南昌航空大學

毕业设计（论文）

题　　目: 基于机器学习的图像版权检测系统

学　　院: 信息工程学院

专业名称: 网络工程

班级学号: 21224232

学生姓名: 朱城源

指导教师: 万孝星

2025 年 5 月

南昌航空大學

毕业设计（论文）任务书

I、毕业设计(论文)题目：

基于机器学习的图像版权检测系统

II、毕业设计(论文)使用的原始资料(数据)及设计技术要求：

1. 通过调研，掌握图像识别与智能检测系统的基本原理（支撑课程目标 6）

2.学习并掌握基于 HTML/CSS/JavaScript 的网页设计方法，结合前端框架 Vue 实现用户友好的人机交互界面。（支撑课程目标 8）

3.利用 Python Flask 框架搭建后端服务，学习接口路由编写、响应机制及 API 安全调用机制，提升前后端分离式开发能力。（支撑课程目标 1）

4.使用 ResNet50 等预训练深度学习模型完成图像特征提取，构建 Logo 与 AI 图像检测模块的核心算法逻辑。（支撑课程目标 3）

5.采用数据结构与算法基础，实现图像特征向量的高效存储与快速相似度计算方法，提高模型响应效率。（支撑课程目标 5）

6.能独立分析图像检测系统在 AI 图像泛滥背景下的实用性、可扩展性和伦理安全问题，增强对系统设计社会影响的理解。（支撑课程目标 2）

7.编写系统使用说明文档与项目部署手册，提升文档规范性和项目交付完整性。（支撑课程目标 4）

8.掌握图像检测过程中多源数据处理方式与融合策略，提升对模型训练与数据增强的理解能力。（支撑课程目标 7）

III、毕业设计(论文)工作内容及进度安排：

1、第 1 周至第 4 周，进行文献调研，制定研究方案，撰写开题报告；

2、第 5 周至第 8 周，按研究方案进行设计和开发，接受毕业设计中期检查；

3、第 9 周至第 12 周，完成设计任务，接受毕业设计终期检查，撰写论文初稿；

4、第 13 周至第 16 周，完成毕业设计论文的修改、答辩和毕业设计资料归档。

Ⅳ、主要参考资料：（5-7 篇为宜）：

[1]高晓杰．图像处理和机器学习实例：学习和应用．中国电力出版社，2018

[2]李建群．深度学习实践：TensorFlow 和 Keras 应用．机械工业出版社，2021 年；

[3]黄晓宇．基于深度学习的商标识别系统．计算机学报，2020 年；

[4]刘安学．图像算法：基础和实现．北京大学出版社，2020 年；

[5] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Deep Convolutional Neural Networks. Advances in Neural Information Processing

[6] College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia. Trademark Image Similarity Detection Using Convolutional Neural Network.

信息工程 **学院** 网络工程 **专业类** 212242 **班**

学生（签名）：

日期： 自 2025 年 01 月 06 日至 2025 年 06 月 01 日

指导教师（签名）：

助理指导教师(并指出所负责的部分)：

网络工程 系（部）主任（签名）：

附注:任务书应该附在已完成的毕业设计说明书首页。

南昌航空大學

学士学位论文原创性声明

本人声明，所呈交的论文是本人在导师的指导下独立完成的研究成果。除了文中特别加以标注引用的内容外，本论文不包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式表明。本人完全意识到本声明的法律后果由本人承担。

作者签名：　　日期：

指导老师签名：　　日期：

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南昌航空大学可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

作者签名：　　日期：

指导老师签名：

基于机器学习的图像版权检测系统

学生姓名:朱城源　　班级:212242

指导老师:万孝星

摘要：随着互联网技术的快速发展，图像数据在各类平台中呈现出爆炸式增长，品牌 Logo 作为企业视觉形象和知识产权的关键元素，频繁地出现在各类数字图像中，随之而来的商标侵权问题也日益严峻。传统的人工比对和专家识别方法在面对大规模图像和复杂变形时效率低、主观性强，难以满足现代图像版权保护的需求。为此，本文设计并实现了一种基于机器学习的图像版权检测系统，聚焦于品牌 Logo 识别与相似度分析，通过机器学习模型提升检测的准确性和自动化水平。

本系统将 ResNet50 作为核心特征提取网络，借助对特征向量做归一化处理，以此保证后续相似度计算的稳定性与可靠性，在相似度评估环节，系统运用余弦相似度方法，精准测量查询图像与数据库中 Logo 的相似程度。为攻克多 Logo 变体识别难题，设计了一种“品牌最大匹配”策略，让系统可准确识别同一品牌下的各类 Logo 样式。

系统功能不光有图像上传与比对，还开发了 Logo 特征库管理、结果可视化展示功能，极大提升了系统的灵活性与可扩展性，在可视化设计方面，提出了一种差异放大策略，融合非线性变换与相对相似度对比图表，切实突出高相似度间的细微差别，帮助用户更精准地判断检测结果。

依靠公开 Logo 数据集开展功能与性能验证，实验显示系统在 Logo 识别精度、处理速度以及用户体验方面表现良好，该系统在技术实现上呈现出创新性，同时拥有实用价值，为商标版权保护提供了一种高效、可扩展的智能化检测手段。

关键词：图像版权检测；Logo 识别；机器学习；ResNet50

指导老师签名：

Machine Learning–Based Image Copyright Detection System

Student Name: ChengYuan Zhu **Class:** 212242

Supervisor: XiaoXing Wan

Abstract: At present, with the rapid development of Internet technology, image data across digital platforms has shown an explosive growth trend. Brand logos are a crucial part of a company's visual image and also a key component of its intellectual property rights. Nowadays, they can often be seen in digital content, which has made trademark infringement increasingly common. The traditional methods relying on manual verification and expert appraisal have encountered challenges in terms of efficiency and objectivity when dealing with large-scale image analysis and complex label changes, and are unable to meet the requirements of modern copyright protection. To solve this problem, a machine learning-driven image copyright detection system, This system mainly focuses on brand identity recognition and similarity analysis, and relies on computational models to improve the accuracy of detection and the degree of automation.

The system uses ResNet50 as the core feature extraction network. By normalizing the feature vectors, the stability and reliability of similarity calculation are guaranteed. During the process of similarity evaluation, cosine similarity measurement can accurately quantify the visual matching situation between the query image and the database entries. To solve the problem of multi-variant logo recognition, a "brand maximization matching" strategy is adopted. This strategy can effectively identify different logo styles under the same brand.

In addition to the basic functions such as image upload and comparison, this system also integrates the logo feature database management and the visualization result display module together. This enhances the flexibility during operation. When conducting visualization processing, the difference magnification strategy combines nonlinear transformation and comparison similarity graphs for use. This approach highlights the subtle differences existing between highly similar matches, which is to provide support for decision-making.

After evaluating the public identification dataset in this paper, it is found that this system performs well in terms of recognition accuracy, processing speed, and user experience. This solution is both technologically innovative and practical, providing an efficient and scalable intelligent tool for trademark protection.

Keywords: Image Copyright Detection; Logo Recognition; Machine Learning; ResNet50

Supervisor's Signature:

目 录

第一章 绪论 1
1.1 研究背景及意义 1
1.1.1 商标侵权案例及其技术困难 1
1.1.2 技术困难的共性分析 3
1.1.3 研究的意义 3
1.1.4 AI 生成图像的版权挑战与检测需求 3
1.2 国内外研究现状 4
1.2.1 国外研究现状 4
1.2.2 国内研究现状 4
1.2.3 国内外研究比较 5
1.2.4 研究空白与不足 5
1.3 主要研究内容 6
1.3.1 Logo 特征提取方法研究 6
1.3.2 Logo 相似度计算算法优化 6
1.3.3 多 Logo 变体识别方法 6
1.3.4 检测结果可视化设计 7
1.3.5 系统整体架构设计与实现 7
1.3.6 AI 图片检测模块 7
第二章 系统分析与设计 9
2.1 需求分析 9
2.2 可行性分析 9
2.2.1 技术可行性 9
2.2.2 经济可行性 9
2.2.3 操作可行性 9
2.2.4 对可持续发展方面的影响 10
2.3 系统架构设计 10
2.4 系统功能描述 11
2.5 系统实现原理 11
2.5.1 ResNet50 网络结构与特征提取 11
2.5.2 图像相似度计算方法 12
2.5.3 品牌最大匹配策略 13

2.6 系统用例 15
2.6.1 Logo 版权检测模块系统用例图 15
2.6.2 AI 生成图片检测模块系统用例图 16
2.7 技术选型 16
第三章 系统实现 18
3.1 开发技术环境 18
3.2 功能模块实现 18
3.2.1 登录注册模块 19
3.2.2 Logo 检测模块 20
3.2.3 AI 图像检测模块 25
3.2.4 Logo 管理模块 27
3.2.5 用户中心模块 29
第四章 关键问题分析与解决方案 31
4.1 特征向量标准化问题 31
4.1.1 问题描述 31
4.1.2 解决方案 31
4.2 多 Logo 变体识别问题 32
4.2.1 问题描述 32
4.2.2 解决方案 33
4.3 高相似度差异显示问题 34
4.3.1 问题描述 34
4.3.2 解决方案 34
第五章 系统测试与评估 37
5.1 功能测试 37
5.2 性能测试 37
5.3 用户体验评估 38
第六章 总结与展望 39
参考文献 41
致谢 43

第一章 绪论

1.1 研究背景及意义

随着互联网以及数字媒体技术的快速发展，图像内容在生产、传播以及使用方面呈现出爆炸式增长的态势，品牌 Logo 属于企业知识产权的关键部分，是企业形象与市场竞争力的核心表现，然而数字化时代的降临为商标侵权行为创造了便利条件，未经授权使用品牌 Logo 的情况屡屡出现。这些侵权行为损害了企业的合法权益，还扰乱了市场秩序，误导了消费者，给予了严重的经济与社会影响，近些年来，商标侵权案件在全球范围内频繁发生，怎样高效且准确地识别与打击侵权行为，变成了企业与法律领域急需解决的重大问题。

传统的商标侵权监测以及认定工作主要依靠人工比对以及专家判断来开展[3]，然而在面对数量众多的数据以及复杂多样的侵权形态时，这样的方法就显得有些力不从心了，鉴于此，开发出一种有自动化特性且高效的图像版权检测系统，其有着关键的学术研究意义，还可为企业给予实用的技术支持。此项研究利用深度学习技术，构建起一个可快速识别商标侵权状况的图像检测系统，用来应对当下商标保护所面临的挑战。

1.1.1 商标侵权案例及其技术困难

为了可更加直观地呈现商标侵权现象的普遍性以及所面临的技术挑战，接下来会列举几个较为著名的商标侵权案例，对其在侵权认定过程中遭遇的技术险阻展开分析，同时阐述借助图像版权检测系统是怎样给予解决的。

1）苹果公司与三星电子在 2012 年有相关纠纷，苹果公司指控三星电子的 Galaxy 系列智能手机和平板电脑侵犯了其 iPhone 和 iPad 的设计专利以及商标权，所涉方面包括外观设计、图标布局以及用户界面元素，法院判定三星要向苹果支付一笔数额不小的赔偿，在侵权认定过程中存在技术险阻，法院得逐个去比对两者的设计细节，像是图标的形状、大小、颜色以及排列方式等。因为对这些元素相似性的判断带有一定主观性，不同专家可能会得出不一样的结论，而且人工比对的工作量极大，在产品种类多、设计更新快的情形下，效率比较低，图像版权检测系统可借助卷积神经网络自动提取设计特征，还可以利用相似度算法比如余弦相似度快速比较两组图像的相似程度。这种办法可以大幅减少人工比对所需时间，还可以给出客观的量化结果，降低主观判断造成的误差，

2）路易威登作为全球知名奢侈品牌，其 Logo 和经典花纹设计长期被仿冒者

复制用来制造低价仿品，这些仿冒品凭借线上线下渠道售卖，严重侵害了 LV 的商标权，仿冒者大多时候对 Logo 做些微小调整，像改变颜色、字体比例或者图案细节，这让人工识别变得特别险阻。另外仿冒品在全球各地分布，数量众多，人工监测很难全面覆盖到，基于机器学习的图像检测系统能训练一个包含 LV Logo 多种变体的特征库，依靠特征提取和匹配技术，就算是很细微的改动也能被识别出来，该系统可以结合网络爬虫技术，自动扫描电商平台和社交媒体上的图像内容，达成全球范围内的侵权监测。

3）耐克在 2019 年指控斯凯奇的鞋类产品侵犯了其 Air Max 270 和 VaporMax 的设计专利和商标权，觉得斯凯奇在鞋底设计和外观上有较高相似性，鞋类产品设计涉及复杂的 3D 结构和多种材料，人工比对要分析设计图纸、实物样品以及照片等多种信息，工作量大且耗费时间。而且相似设计的判断容易受专家主观经验影响，致使结果不一致，图像版权检测系统支持多角度图像输入和 3D 模型分析，凭借深度学习模型提取鞋类设计的关键特征比如鞋底纹路、轮廓形状等，实现高精度的相似度比对，这种自动化方式可减轻人工负担，还可以提高判定的准确性，

4）阿迪达斯与 Forever 21 在 2017 年产生相关纠纷，阿迪达斯指控 Forever 21 在其服装产品上采用了和自己标志性“三条纹”Logo 相近似的设计，觉得这样的行为构成了商标侵权，“三条纹”Logo 因为设计简单，容易被模仿或者改编，在依靠人工判断相似设计的时候大多时候会有争议。服装市场更新速度快，产品种类又多，导致人工监测很难跟上步伐，图像检测系统可依靠训练来识别“三条纹”Logo 的核心特征，像条纹数量、间距、角度等，还可以自动扫描市场上的服装产品，找出潜在的侵权行为，该系统还可以生成可视化报告，直观地呈现相似度分析结果，辅助法律方面做出决策。

5）星巴克与小型咖啡店 Logo 侵权有不少起相关案件，星巴克的圆形 Logo 辨识度高，大多时候被小型咖啡店或者饮品店模仿使用，这些模仿者可能只是在颜色或者细节上稍微做些改变，想要避开侵权指控，小型商家的 Logo 设计多种多样，而且分布得很零散，人工监测没办法覆盖所有潜在的侵权行为。另外部分模仿只涉及局部特征，这就增加了识别的难度，图像版权检测系统可构建一个包含星巴克 Logo 及其变体的数据库，借助图像识别技术[17]扫描网络上以及实地拍摄的图像，快速找到疑似侵权的 Logo，就算是局部模仿，系统也能借助特征匹配技术准确识别出来。

1.1.2 技术困难的共性分析

依靠对典型商标侵权案例进行深入剖析，可以总结出在认定过程中常见的技术障碍，人工比对受到专家主观判断的限制，不同专业人员的评估标准存在诸多差异，这容易引发结果争议，市场上产品设计的种类繁多且数量庞大，逐个进行比对会耗费大量时间且效率低下，难以契合海量数据以及快速变化的市场需求。侵权者大多时候依靠微调或变形 Logo 来规避检测，这增加了识别的难度，人工很难精准辨别其中的细微变化，产品设计的快速更新以及全球市场的流通，使得人工监控难以同步，无法做到全面覆盖，机器学习的图像版权检测系统为应对这些挑战提供了有效的解决办法，该系统利用深度学习模型 ResNet[4]自动提取 Logo 的高级语义特征，减少了人为干预，提升了检测的客观性。借助高效算法计算图像相似度，可迅速处理大规模的比对任务，适应海量数据的场景，系统支持识别同一品牌下的多种 Logo 变体，提高了检测的稳健性和精确度，有效解决了变形识别的难题，结合网络爬虫与图像处理技术，实现了对线上线下内容的实时分析，适应了市场快速变化的节奏，为商标侵权监测提供了精准可靠的技术支持。

1.1.3 研究的意义

本研究有多方面的关键价值，所研发的高效图像版权检测工具可自动识别并监控品牌 Logo，可企业保护知识产权，降低因侵权行为导致的经济损失以及品牌形象受损的风险，该系统采用量化相似度分析方法，同时结合直观的可视化呈现方式，减少了侵权认定过程中的主观偏差，为法律判定提供客观依据，提升了判定的公正性与可信度。其自动化检测机制极大地节省了人工监控所需的时间和精力，大幅扩大了侵权识别的覆盖范围，加快了响应速度，以契合现代商业环境对效率的迫切需求，研究将深度学习与计算机视觉技术引入商标保护领域，拓展了人工智能在法律及商业场景中的应用，呈现出技术创新的驱动作用，借助有效遏制商标侵权行为，该系统维护了消费者权益，促进了公平竞争市场氛围的形成，可规范化商业生态的构建，为企业和消费者营造了更优越的环境。

1.1.4 AI 生成图像的版权挑战与检测需求

近些年来人工智能技术的发展速度较快，这促使图像生成技术得到广泛应用，基于生成对抗网络以及扩散模型的虚拟图像生成技术，可创造出十分逼真的图像，一些不法分子可能会利用借助 AI 生成的这些图像伪造品牌 Logo、产品图片[13]，或者进行虚假宣传，实施商标侵权以及知识产权侵害行为。比如在电商平台上，

利用 AI 合成的虚假图像可能会被用于仿冒品牌商品，误导消费者，损害企业声誉，与传统侵权识别相比，因为 AI 生成图像有高度仿真性，传统的人工目视检查或简单的特征比对方法很难有效辨别，为应对这一挑战，图像版权检测系统需要引入专门的 AI 图像识别模块，作为潜在侵权风险的初步筛选工具。该模块会对噪声分布、纹理特性、边缘异常以及色彩分布等特征进行分析，快速自动辨识出 AI 生成图像，阻止合成内容进入后续版权检测流程，提升系统对新型侵权风险的防范能力，这一举措降低了人工筛选成本，提高了系统应对新兴侵权手段的能力，体现了知识产权保护技术的前瞻性发展。

1.2 国内外研究现状

1.2.1 国外研究现状

国外在图像版权检测、Logo 识别以及相似度分析这些领域展开研究时，广泛运用深度学习技术，近些年来，卷积神经网络也就是 CNN，在 Logo 检测与识别方面有着较为广泛的应用，取得了一定成果，Hou 等人所做的综述显示，深度学习技术依靠其出色的特征表达能力，拥有更高的精确度以及处理速度，受到学术界的关注[25]。从 2015 年开始，基于深度学习的 Logo 检测方法不断出现，主要探索怎样自动提取图像特征，以此来实现 Logo 的精准识别与定位，商标图像相似度检测也是研究的热门方向[10]，学者们大多时候借助预训练网络，像 ResNet－50 或者其变种，来提取图像特征向量，再结合最近邻搜索等技术进行相似度匹配。比如 Showkatramani 等人运用 ResNet－v2 模型提取商标特征，借助最近邻搜索完成相似图像检索[26]，另外有部分研究引入孪生网络也就是 Siamese Network 与深度哈希技术，把查询图像和数据库图像映射到特征空间，计算哈希距离来评估相似性。这些国外研究大多集中在提升检测精度上，大多时候构建复杂的深度网络模型，例如 ResNet、VGG，并在公开数据集上验证性能，凭借优化网络架构与损失函数不断改进效果。

1.2.2 国内研究现状

国内于图像版权检测领域展开探索时，更倾向于关注模型轻量化以及系统化应用方面，为了可契合工程化需求以及实时检测场景的要求，研究者们一直积极希望能够开发轻量级网络或者对大型网络模型进行压缩，举例来说，有部分研究选用 MobileNetV2 模型来开展特征提取工作，该模型有参数数量较少、计算量较低的特点，适合在资源受限的环境中使用。国内的团队同时还对模型蒸馏与剪枝

技术颇为重视，期望凭借这些技术来缩小模型规模并且提升推理速度，另外国内的研究大多集中在内容安全与版权保护的实际应用场景，一些大型互联网企业把检测技术融入到实际平台当中，就像阿里云内容安全平台借助目标检测技术来识别图像与视频里的 Logo、商标、水印等元素，提供类别、名称以及位置等信息，以此服务于品牌监控与版权保护工作。在系统设计层面，有部分研究构建起了较为完善的本地图像检索系统[7]，借助深度学习模型提取特征后结合向量搜索引擎来达成高效检索，比如张浩东等人所开发的系统运用 Triton Inference Server 来部署深度模型，搭配 Milvus 向量搜索引擎，可支持大规模特征存储以及快速查询。这些成果呈现出国内研究从算法优化直至工程落地的完整路径，重视算法性能，也对系统部署与实际应用给予关注，国内研究在模型轻量化以及端到端检索系统的开发方面取得了一定进展，体现出较强的应用导向以及扩展能力。

1. 2. 3 国内外研究比较

研究思路方面，国外的研究大多聚焦于算法开发以及模型性能的优化，借助运用更深层次或者复杂的网络架构，像 ResNet、ResNet-v2、孪生网络等，以此来提高检测的精度，与之形成对比的是，国内的研究则更倾向于把成熟的算法融入到实际的系统当中，更加注重端到端系统的性能，在模型轻量化以及工程化部署等方面都有所涉及。技术难点方面： 国外工作面对的是多样化的侵权样本和大规模数据库，因此难点在于提高模型对各种变形、遮挡等复杂场景的鲁棒性；国内工作更关注如何在资源受限（如移动端、实时服务）环境下保证检测速度和精度，难点在于模型压缩、加速推理以及完整检索系统的构建。应用场景方面：国外研究多服务于版权管理、品牌保护和学术研究场景，常在公共数据集上验证技术；国内应用场景则多与在线内容审核和产业化部署相关，如电商平台的竞品监测和社交媒体的版权监测，强调系统的可用性和业务效果。国外多关注算法本身，国内则兼顾算法和应用环境的匹配。

1. 2. 4 研究空白与不足

综合审视国内外已开展的相关研究，在图像版权检测以及相似度检索这两个领域已经取得了一些进展，深度学习技术让 Logo 识别以及图像匹配的精度有了提升，工程化系统可以达成实时检测与检索，不过当前的工作依旧存在一些局限以及研究空白，多数方法依靠预训练的深度网络，模型复杂且计算需求高，不太容易直接应用于实时系统或者移动设备。虽然内容驱动的相似图像检索系统在国

内外都有研究探索，但是专门为版权检测设计的综合性系统比较少，大多时候缺少对大规模图像库的检索能力以及多模态分析功能，比如图像结合文本这种情况，对于复杂侵权场景，像部分篡改或者仿制图像，现有的检测能力还不够，国内研究在模型轻量化以及系统化应用方面有了突破，但是怎样平衡高精度和高效率还需要研究。本研究针对上述这些不足，开发了基于 ResNet50 的轻量化图像相似度检测系统，来在保持高检测精度的同时实现快速检索以及工程落地，弥补现有研究在效率和实用性方面的不足。

1.3 主要研究内容

本论文以《基于机器学习的图像版权检测系统》为题，围绕系统核心功能与关键技术展开，主要研究内容可分为以下六个方面。

1.3.1 Logo 特征提取方法研究

为了应对 Logo 图像在拍摄角度、尺度以及光照等多种条件下呈现出的多样性状况，本研究专门开发了一套有稳定性的特征提取流程，具体选用 ResNet50 作为核心网络，借助迁移学习的方式加载预训练权重，并且在 Logo 数据集上展开微调操作，以此来契合对于品牌标识细节提取的相关需求。研究着重关注特征向量标准化这一问题，采用 L2 范数归一化处理手段，消除因不同图像特征数值尺度存在差异而引发的匹配误差，在提取流程当中融入异常向量检测机制，保证特征质量稳定且可靠，可适用于后续的相似度计算工作。

1.3.2 Logo 相似度计算算法优化

在 Logo 比对模块里，本研究设计并且比较了好些相似度度量方法，提出一种“品牌最大匹配”策略，针对同一品牌之下的多种 Logo 变体，系统逐个计算查询图像与各个变体的相似度，挑选出最高得分当作该品牌的最终匹配结果，以此来达成多样式 Logo 的全面识别，为了提升算法对高相似度样本的区分能力，系统引入动态阈值调节机制，放大接近分值之间的细微差别，并且辅以差异化分级提示，帮助用户更加精准地评估相似度结果。

1.3.3 多 Logo 变体识别方法

为了适应同一品牌的 Logo 在不同场景或者时期，由于颜色、形状以及细节方面的变化而出现的多种变体情况，本研究针对传统单一特征库结构展开优化工作，设计出了品牌—Logo 映射的数据结构，把每个品牌名称与包含其所有变体

特征向量的列表建立关联，当新的 Logo 样本被导入时，系统会自动解析品牌及其所属变体，批量提取特征并存储到数据库当中。在识别阶段，系统借助比对列表里的所有特征向量并筛选匹配项，达成对品牌多样化 Logo 的全面检测。

1.3.4 检测结果可视化设计

此次研究设计了多种可视化展示方案，来让用户可以更直观地解读比对结果，除了常规的相似度排名表之外，还开发了以下功能，其一为差异放大折线图，该图是在原始相似度曲线的基础上，叠加经过非线性变换处理后的差异曲线，以此突出高分值之间的细微差距，其二是水平条形图，其采用高起点截断 Y 轴的设计方式，可清晰呈现各品牌匹配度之间的微小差异。其三是相对差异对比表，此表并列展示查询 Logo 与最佳匹配品牌的数值差以及百分比，并且配有直观进度条，可提供清晰的匹配差异提示，这些功能借助 Chart.js 等前端可视化库得以实现，同时支持导出 PNG 格式报告，便于在法律取证与评审流程中应用。

1.3.5 系统整体架构设计与实现

论文从系统工程角度出发，设计了基于前后端分离的三层架构：

1）表示层：采用 HTML5、CSS3 及 JavaScript 框架，实现了响应式的用户界面，支持 Logo 上传、检测结果预览和历史记录管理；

2）业务逻辑层：基于 Python Flask[6]框架，封装了特征提取、相似度计算和可视化数据接口，提供 RESTful API 服务；

3）数据访问层：Logo 基础信息存储于 JSON 格式文件中，特征向量使用 Pickle 序列化存储，并借助内存索引加速读取操作。

系统还提供特征库管理与重置功能，用户可在线增删 Logo 样本并一键更新特征库，保证系统长期运行的稳定性与可维护性。

1.3.6 AI 图片检测模块

随着人工智能生成图像技术不断向前发展，AI 合成的 Logo 以及图像伪造情况给传统版权保护给予了全新的挑战，这类内容由于有高度仿真的特性以及一定的隐蔽性，使得人工鉴别以及依据规则驱动进行检测的难度有所增加，为了应对这一问题，本研究在图像版权检测系统中引入了 AI 图像识别模块，将其作为潜在侵权内容的前置筛选环节，针对用户上传的图像开展来源分析以及风险评估。该模块依靠多维度特征分析来判断图像是否由 AI 生成，其核心技术涉及以下几个方面，其一噪声分布检测，即对图像中随机噪声的细微特征进行分析，以此捕

捉 AI 生成可能留下的伪影痕迹，其二纹理与边缘分析，主要检查图像纹理细节以及边缘轮廓的连续性，识别生成算法可能导致的异常平滑或者失真情况。其三色彩分布评估，着重审查图像整体色彩过渡以及分布规律，辨别不符合自然拍摄特征的颜色异常现象，其四元数据核查，依靠验证图像的基础属性，如透明通道、文件格式或者分辨率是否存在异常，借助元数据线索辅助判定，依靠这些技术手段，AI 图像检测模块可在图像进入侵权比对之前识别出疑似 AI 生成的内容，发挥提前过滤以及预警的作用。该模块的应用价值在于提升系统应对 AI 合成内容等新型侵权形式的能力，与主系统功能形成有效的协同配合，为图像版权保护提供更为全面的保障。

综合来看，本论文结合图像版权检测的实际需求，系统地开展了从特征提取、相似度计算、多变体识别到可视化展示与系统部署的全流程研究并给予实现，为高效且精准的 Logo 侵权检测以及 AI 图片检测提供了完整的解决方案。

第二章 系统分析与设计

2.1 需求分析

当下市场里图像侵权问题不断出现，企业的商标以及品牌 Logo 被恶意仿冒、滥用的情况十分常见，传统的人工版权监测方法效率不高且主观性较强，很难契合网络时代大量图片的检测要求，随着 AI 生成技术快速发展，借助人工智能生成逼真图片来实施侵权的风险也慢慢显现出来。用户急切需要一个自动化、高精度、可快速反应的版权检测系统，该系统要集成基本的 Logo 侵权检测以及 AI 生成图像识别功能，系统要有用户友好的图形界面，方便用户快速上传图片并获得详细、直观的检测分析报告，用户管理和 Logo 特征库管理也得高效且易于维护。

2.2 可行性分析

2.2.1 技术可行性

本系统有坚实且成熟的技术根基，它所采用的深度学习技术，确切地说是 ResNet50 网络，在图像特征提取以及分类方面有着广泛的运用[9]，相关模型的预训练权重以及算法库的支持相当完善，基于 Python Flask 构建的后端架构也同样得到了广泛的验证[11]，有良好的稳定性与可扩展性，而且社区资源丰富，可迅速解决开发过程中的技术难题。前端界面设计采用了 HTML5、CSS3 以及 JavaScript 框架，这些技术在各类商业项目里已被广泛运用，有着丰富的文档资料与实例代码，方便进行开发与维护，技术实施难度处于适中水平，开发风险较小，技术架构稳固且可靠。

2.2.2 经济可行性

从经济层面展开分析可知，本系统所运用的软件开发工具以及技术框架皆属于开源免费资源，如此一来便切实降低了前期的经济投入，该系统对于服务器资源的需求较低，在初始阶段，仅仅需要适度的云服务器资源便可支撑业务需求，成本处于可控范围，随着未来用户数量的增长以及业务需求的扩展，服务器规模可依据实际需求进行动态扩展，按需付费的模式可优化成本。另外该系统自动化程度较高，可大幅减少企业在版权监测方面的人力投入，从长远来看有一定的经济优势。

2.2.3 操作可行性

系统从设计开始就充分顾及了用户操作的便利性，系统界面清晰又简洁，功

能布局很合理，运用图形化且直观化的操作形式，就算用户没有专业的技术背景也可轻易完成操作流程，举例来说，用户只要凭借简单的“选择文件”按钮上传要检测的图片，系统会自动开展 Logo 侵权比对以及 AI 生成检测，并且以图表、分析报告等形式直观地给用户呈现检测结果。特征库管理提供单个导入和批量导入这两种方式，用户可轻松维护与更新 Logo 数据，用户管理模块提供了清晰的个人资料查看与修改界面，切实提高了操作的便捷程度以及用户满意度。

2.2.4 对可持续发展方面的影响

此系统运用云端部署方式，资源依照需求来使用，避免了本地服务器资源出现闲置状况以及能源被浪费，可切实降低系统运行所产生的碳排放，系统的应用也可切实打击网络上的品牌侵权行为，维护公平竞争的商业环境，促使知识产权保护意识得到提升，这对经济的良性循环有益，并且对社会和经济的可持续发展有着长期积极影响。

2.3 系统架构设计

系统运用经典的三层架构模式，把整个应用划分成了表示层、业务逻辑层以及数据层，前端表示层专门负责与用户展开交互，借助浏览器界面来呈现信息并且收集用户输入，当用户在前端发起操作的时候，前端会经由 HTTP 请求把数据发送给后端，后端逻辑层是由 Flask 框架搭建而成的，承担着主要的业务逻辑处理工作，覆盖路由转发、权限验证、AI 模型推理等方面。针对不同的请求，后端会调用相应的模块：要是登录请求，就去访问数据库校验用户信息，要是图像检测请求，就调用 TensorFlow 深度学习模型进行处理，后端加载了预训练的图像识别模型，先对用户上传的图片进行预处理，接着利用模型执行 Logo 识别或者图像内容分类，得到分析结果之后把它封装成响应数据返回前端页面展示。数据层用于对系统运行所需的数据进行持久化存储和管理，包含关系型数据库和文件存储两部分，系统使用浏览器 localStorage 轻量级数据库来保存结构化数据，像用户账号、Logo 信息、检测记录等，同时采用文件方式保存模型和图像等非结构化数据，例如训练好的模型权重文件以及用户上传的原始图片和结果标注图片等。在该三层架构中，表示层和数据层不直接交互，所有通信均通过后端业务逻辑层完成。这种设计保证了各层职责单一，前端专注于界面交互，后端负责业务和算法处理，数据层管理持久数据，从而提高了系统的灵活性和可维护性。

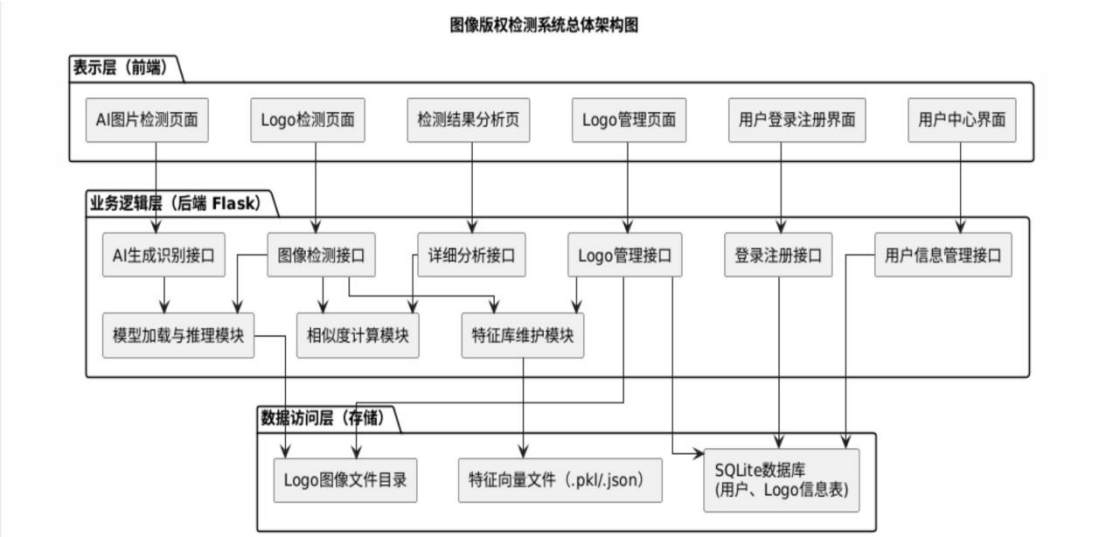

图 2-1 系统框架图

2.4 系统功能描述

本系统整合了图片版权检测、AI 生成图片检测、Logo 数据库管理以及用户管理这四大关键功能模块。

在这之中，图片版权检测模块借助用户上传待检测的图片，自动调用后端的 ResNet50 模型来开展特征提取以及相似度计算工作[14]，精准地识别并定位图片与 Logo 库中最为匹配的商标图案，同时提供丰富多样的检测结果可视化呈现，像是相似度排名柱状图、饼图、趋势折线图、详细排名表以及相对相似度差异分析表格等。

AI 生成图片检测模块运用 OpenCV 实现噪声模式、纹理、边缘以及颜色分布的分析，快速判断图片是否由 AI 生成，并给出详尽的分析依据。

Logo 数据库管理模块让用户可方便地进行单个 Logo 的新增与维护工作，并且也支持批量导入操作，系统会自动提取并存储 Logo 特征，还提供一键特征库重置与更新功能，以此保障数据库维护的效率和准确性。

用户管理模块有用户注册、登录、个人信息修改以及上传历史记录查看等功能，个人中心界面布局较为直观，用户信息和操作记录清晰明了，提升了用户操作的便利程度与体验感受。

整个功能体系紧密依据用户需求构建而成，功能模块之间相互联动协调，可充分契合企业与用户在图像版权保护领域的实际需求。

2.5 系统实现原理

2.5.1 ResNet50 网络结构与特征提取

ResNet50 属于深度卷积神经网络模型，该模型一共有 50 层，其关键创新点是引入了残差模块也就是 Residual Block，残差模块是在堆叠的卷积层之间添加跳跃连接，把输入信号也就是 x 和卷积层的输出即 F(x)直接相加得出最终输出 H(x) = F(x) + x。这个简单的恒等映射操作如同在网络里构建了一条“捷径”，可让信息以及梯度跳过部分网络层直接传递，它成功解决了深层网络训练时的梯度消失问题，让网络可堆叠更多层而不会遇到训练险阻，残差连接使每个模块只需学习输入和期望输出之间的“差值”部分也就是残差，要是这个差值为零，网络就能直接学得恒等映射，以此保证即便网络加深也不会降低性能。这种设计使 ResNet50 在保持高性能的情况下大幅加深了网络深度，较大提升了对图像特征的表达能力。

ResNet50 运用瓶颈残差块结构以降低参数量，每一个残差块里面有 3 个卷积层，先是经 1×1 卷积来降低维度也就是减少通道数，紧接着运用 3×3 卷积提取特征，最后再借 1×1 卷积提升维度即恢复通道数，这种“先降后升”的瓶颈设计相较于直接运用高维度的 3×3 卷积，很大程度减少了计算量以及参数数量。借助瓶颈块，ResNet50 在保证网络深度的时候把参数规模控制在大约 2500 万，在网络层次结构方面，ResNet50 可分成多个阶段，最开始是一个没有残差块的卷积层和池化层，用来初步提取特征，之后连续四个阶段堆叠残差块，各阶段残差块数量依次是 3、4、6、3 个，逐步让网络深化。随着阶段推进，特征图的空间尺寸渐渐变小、通道数渐渐增多，最终在第五阶段输出尺寸为 7×7、通道数 2048 的特征图，凭借在末端对特征图做全局平均池化，网络得到一个长度为 2048 维的图像特征向量，这个高维特征向量凝聚了输入图像的关键信息，可用来表示图像的视觉特征。

ResNet50 网络足够深且经过大型数据集预训练，其提取的特征判别性强，适用于图像特征提取任务，本系统采用预训练的 ResNet50 模型进行特征提取，处理输入的 Logo 图像，将其转换为 2048 维特征向量表示，如此每张图像都被编码为一个特征向量，覆盖图像中 Logo 的形状、纹理等高层语义信息。凭借 ResNet50 强大的特征提取能力，系统能获得稳定且有区分度的图像表示，为后续相似度比对奠定基础，ResNet50 的残差网络结构能保证足够深度以提取丰富特征，又凭借合理模块设计控制模型复杂度，是本系统图像特征提取的理想选择。

2.5.2 图像相似度计算方法

当获取到图像的特征向量之后，系统需要一种可对任意两张图像的相似程度

给予度量的算法，此系统选用了余弦相似度当作图像特征比对的度量手段，余弦相似度乃是借助计算两个向量夹角的余弦值衡量它们相似性的一种指标，其具体的计算公式如下：

$$S_{im}\cos(A,B)=\frac{A\cdot B}{\|A\|\cdot\|B\|} \tag{1.1}$$

其中$\|A\|\cdot\|B\|$表示向量|A|和|B|的点积，$\|A\|$和 $\|B\|$分别表示向量的长度。余弦相似度的取值范围为[-1，1]：当两个向量方向完全相同时，余弦相似度为 1；若方向正好相反，则相似度为-1；若垂直则为 0。该相似度关注的是向量方向上的相似程度，而不受向量长度尺度影响。也就是说，即使两个特征向量的绝对大小有所不同，只要它们指向的方向一致，余弦相似度仍会取得很高的值。这种性质使其非常适合比较图像特征，因为不同图像的特征向量可能因亮度、尺寸等因素造成整体幅值差异，但方向相似仍能反映出内容上的相近。

在系统实现阶段，针对需要进行比较的两张图像，分别提取出各自的特征向量，之后计算这两个特征向量之间的余弦相似度，以此来判断图像之间的相似程度，当相似度越趋近于 1 时，意味着两幅图像在特征空间里的方向越发相近，视觉内容也更为相似，而当相似度接近 0 或者为负数时，则说明两者几乎不存在相似的特征。比如要是输入图像中的 Logo 和数据库里的某一 Logo 模板十分相似，那么它们对应的特征向量的余弦相似度或许会超过 0.9，这就说明两图相似度很高，相反要是输入图像与另一张不相关图像的特征向量余弦相似度仅为 0.1，那就说明它们在特征空间里毫无接近度，可判断为不同的内容。凭借设定合适的相似度阈值，系统可依据此来判定两张图像是否属于同一品牌或者有相同的标志元素，即当相似度高于阈值时判定为匹配，低于阈值则判定为不相似，余弦相似度提供了一种直观且有效的途径来衡量图像特征之间的差异，在本系统的图像比对环节中发挥着核心作用。

2.5.3 品牌最大匹配策略

鉴于实际情形中同一品牌有可能存在多个有不同风格或者版本的 Logo，此系统设计了“品牌最大匹配”策略，以此提升识别的精准度，展开来说，在数据库里为每个品牌存储该品牌之下所有常见的 Logo 样式的特征向量，当有新的待检测图像输入进来时，系统先提取其特征向量，接着依次同该品牌的所有 Logo 特征开展相似度计算。换言之，针对每个品牌，都会获取一组相似度分值，分别对应“输入图像与该品牌各个参考 Logo”的相似程度，随后从这组相似度里挑

选出最大的那个值，当作该品牌对于当前输入图像的匹配度评分，这个过程就如同询问："在某品牌的所有标志模板当中，哪一个与输入图像最为相似？"选取最大值便能衡量出输入图像与该品牌的最佳匹配程度。举例来说，假定品牌 A 在库中有两种 Logo 样式，要是输入图像中的标志恰好是品牌 A 的样式 2，那么该图像与样式 2 模板的余弦相似度或许能达到 0.95，而与样式 1 模板的相似度仅有 0.50，依照最大匹配策略，选取 0.95 作为品牌 A 的匹配分数。如此一来，就算同一品牌的其他 logo 样式与输入图像不太相像，只要有一种样式高度匹配，品牌 A 总体上依旧会得到一个较高的相似度评分。

随后系统会针对所有品牌的最高匹配分值展开比较，从中挑选出分数最高的品牌当作识别结果，一般情况下，分数最高并且超过预设阈值的品牌会被判定为输入图像所属的品牌，要是最高分依旧低于阈值，那就会认定输入图像不包含数据库里的已知品牌 Logo，以此防止出现错误匹配。通过这种"胜者为王"的选择机制，系统能够从众多候选中挑选出与输入图像最相似的品牌标志。继续以上例子，若品牌 A 的最高相似度 0.95 高于其他任何品牌，而且也高于系统识别所需的阈值，那么系统将识别输入图像为品牌 A 的 Logo。这样，当输入图像包含某个品牌的任一合法 Logo 时，该品牌都能因为其中某个模板的高匹配度而被正确识别出来。

"品牌最大匹配"策略于本系统中有着关键效用：其一，它化解了一品牌多 Logo 情形下的匹配难题，可防止系统因参考模板的局限而出现漏检状况，举例而言，就算品牌启用了全新设计的 Logo，只要事先把它的特征录入数据库，当输入图像匹配到这个新 Logo 时就能提升品牌的匹配评分，保证新旧 Logo 均在识别范畴内。其二该策略借助选取最大相似度而非平均相似度来表征品牌，只要有一个 Logo 样本与输入相近，品牌便不会被低相似度的其他样本所影响，此举措提升了识别的召回率与精度：保证了每个品牌的不同 Logo 风格都被纳入考量，提高了识别全面性，还凭借阈值判断规避了偶然高分的误匹配，维持了识别可靠性。"品牌最大匹配"策略切实整合了同品牌多样式 Logo 的特征比对结果，保障本系统在复杂多变的商标标志识别场景中依旧可精准判定图像所属的品牌，为图像版权检测提供了完善可靠的技术支撑。

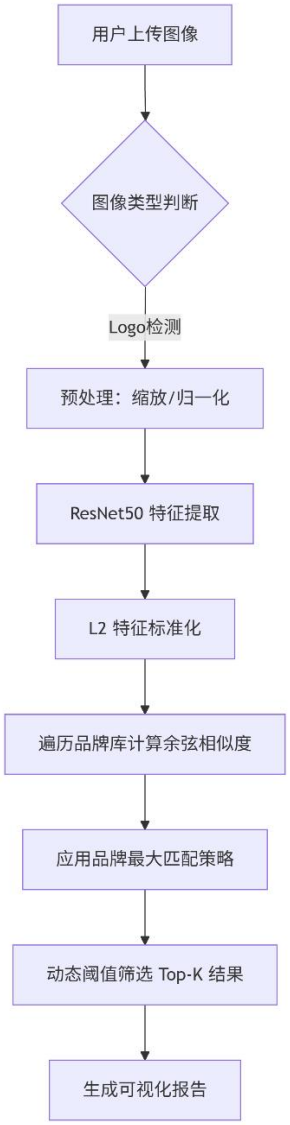

图 2-2 系统工作原理

2.6 系统用例

系统用例图的作用是描述用户与系统之间的交互行为，它可将系统提供的功能模块以及用户的操作边界清晰地呈现出来，在本系统里，用户是企业单位或者其操作人员，直接利用平台开展图像版权检测以及 AI 图片生成识别等功能操作，系统的整体功能可以划分成两个核心业务模块，分别是 Logo 版权检测模块和 AI 生成图片检测模块。下面会分别对这两个模块的系统用例图给予说明。

2.6.1 Logo 版权检测模块系统用例图

Logo 版权检测模块属于系统的核心功能部分，主要负责判定用户上传的图片跟 Logo 特征库内的注册商标有无相似状况，识别潜在的侵权行为，企业用户可上传待检测图片，系统会自动开展特征提取工作，并将其与 Logo 数据库里的向

量特征做相似度匹配，最终生成可视化的分析结果，其中涉及相似度图表、匹配排名以及和其他 Logo 的相似性差异等内容。图 2-3 呈现出了该模块的系统用例图，企业用户与系统交互的主要操作有上传图片、自动开展 Logo 特征提取、进行相似度匹配，还可查看分析图表、相似 Logo 列表以及排名详情，此用例图较大体现了 Logo 检测模块从输入到多维度输出的完整交互过程。

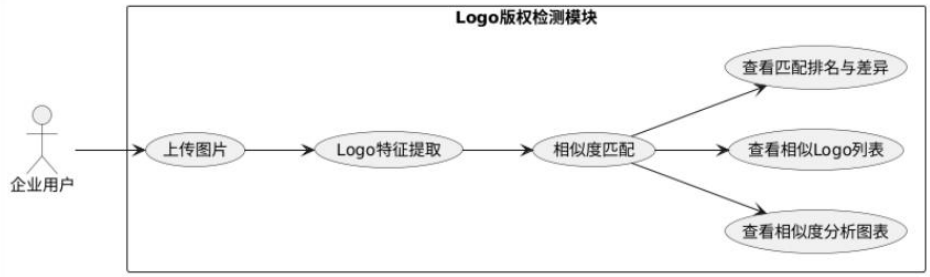

图 2-3 Logo 版权检测模块系统用例图

2.6.2 AI 生成图片检测模块系统用例图

AI 图片检测模块属于系统于图像版权检测功能之上的关键扩展部分，其主要作用为辨别上传的图片是不是借助人工智能技术生成的，以此避免 AI 生成的图像被运用到伪造商标或者虚假侵权行为当中，此模块借助多维度的图像特征分析，再结合图像的基础属性，去计算图片是由 AI 生成的概率，并且给出检测结论以及分析依据。

图 2-3 呈现出了 AI 生成图片检测模块的系统用例图，企业用户上传图片之后，系统会同时启动多项图像分析任务，最后汇总输出检测概率以及结果报告，帮助用户对图像来源进行可信度方面的判断，该模块提升了系统针对新型图像生成手段的识别以及防范能力。

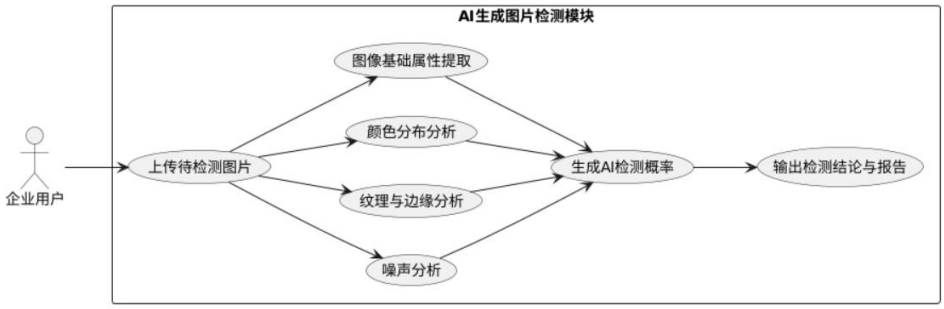

图 2-4 AI 图片检测模块系统用例图

2.7 技术选型

本章着重介绍系统开发所需的主要技术工具及其应用场景，为系统功能实现提供技术支持。

后端框架：Flask（用于搭建 Python 后端 Web 服务，负责业务逻辑与模型调用）[16]

深度学习框架：TensorFlow（用于加载并推理预训练模型 ResNet50，实现图像识别与特征提取）

图像识别模型：ResNet50（用于提取图像深层特征，识别图像中 Logo 的语义类别）

数据库：localStorage、Pickle、JSON（用于存储用户信息、检测记录及 Logo 样本数据）

前端框架：HTML5，CSS，JavaScript（用于构建系统界面，实现交互与数据展示）

图像处理工具：OpenCV（用于图像预处理，如缩放、格式转换及标记绘制）

数据格式支持：JSON ／ Pickle（JSON 用于配置与前后端数据传输，Pickle 用于模型与特征的序列化存储）

第三章 系统实现

在这一章节之中，将会聚焦于开发环境配置以及功能模块实现这两个要点，对图像版权检测系统的具体实现流程给予详细的介绍，一开始会先对系统开发环境展开阐述，其中包括后端框架、深度学习库、数据库环境以及前端构建工具等方面，清晰地说明各类技术工具在开发进程里所发挥的作用以及相应的部署方式。接下来，会把系统划分成若干个关键功能模块，像用户登录、Logo 图像检测、AI 生成图片识别、特征库管理以及检测结果展示等，按照顺序呈现这些模块的页面界面以及功能交互流程，并且结合核心代码截图对关键功能的技术实现方法进行讲解。

3.1 开发技术环境

项目的主要开发环境和所用技术如表 3-1 所示。

表 3-1 项目开发环境与技术栈

类别	技术平台	说明
开发语言	Python	使用 Python 作为后端开发语言，编写服务器逻辑和机器学习算法代码。
后端框架	Flask	采用 Flask 轻量级 Web 框架搭建服务器，提供网页路由和 API 接口服务。
深度学习框架	TensorFlow	基于 TensorFlow 深度学习框架实现模型训练与推理，调用 GPU 加速图像识别计算。
图像识别模型	ResNet50	50 层卷积神经网络模型（残差网络），预训练于 ImageNet 数据集，作为本系统的 Logo 及通用图像识别模型。
数据库	localStorage	嵌入式关系数据库，以单一文件形式存储数据，用于保存用户信息、Logo 记录等。
前端技术	HTML/CSS/JavaScript	使用 HTML、CSS 和 JavaScript 编写前端页面，提供用户界面展示和交互功能。
图像处理库	OpenCV	使用 OpenCV 库进行图像预处理（如缩放、格式转换）以及绘制识别结果标注等操作。
数据文件	JSON、Pickle	JSON 用于配置和前后端数据交换；Pickle 用于序列化保存模型和特征数据，供后端快速加载。

3.2 功能模块实现

系统主要包括登录注册、Logo 检测、AI 图像检测、Logo 管理、用户中心五大功能模块。各模块的实现逻辑简要说明如下。

3.2.1 登录注册模块

提供用户账户的注册与登录功能。注册时，用户通过前端表单提交用户名、密码等信息，后端对密码进行哈希加密后写入浏览器 localStorage 数据库保存；登录时，后端根据用户提交的凭证查询数据库比对验证，成功则建立会话状态以维持登录。页面如图 3-1 所示。

图 3-1 用户注册与登录界面

用户注册与登录核心代码如下：

```
// 注册表单提交处理
registerForm.addEventListener('submit', function(e) {
    e.preventDefault();

    const username = document.getElementById('username').value.trim();
    const password = passwordInput.value;
    const confirmPassword = confirmPasswordInput.value;
    const email = document.getElementById('email').value.trim();

    // 获取现有用户数据
    let users = [];
    try {
        const usersData = localStorage.getItem('users');
        users = usersData ? JSON.parse(usersData) : [];
    } catch (error) {
        console.error('解析用户数据出错:', error);
        users = [];
    }

    // 检查用户名是否已存在
    const existingUser = users.find(u => u.username === username);
    if (existingUser) {
        showAlert('用户名已存在，请选择另一个用户名');
        return;
    }

    // 添加新用户
    const newUser = {
        username: username,
        password: password,
        email: email,
        registerDate: new Date().toLocaleDateString('zh-CN'),
        location: '',
        bio: ''
    };

    users.push(newUser);
    localStorage.setItem('users', JSON.stringify(users));
});
```

图 3-2 注册功能核心代码

```javascript
// 登录表单提交处理
loginForm.addEventListener('submit', function(e) {
    e.preventDefault();

    const username = usernameInput.value.trim();
    const password = passwordInput.value;
    const rememberMe = document.getElementById('rememberMe').checked;

    // 获取用户数据
    let users = [];
    try {
        const usersData = localStorage.getItem('users');
        users = usersData ? JSON.parse(usersData) : [];
    } catch (error) {
        console.error('解析用户数据出错:', error);
        users = [];
    }

    // 查找匹配的用户
    const user = users.find(u => u.username === username && u.password === password);

    if (user) {
        // 登录成功处理
        const token = 'login-token-' + Date.now();
        localStorage.setItem('currentUser', JSON.stringify(user));
        localStorage.setItem('userToken', token);
        localStorage.setItem('lastActivity', Date.now());
    }
});
```

图 3-3 登录功能核心代码

3.2.2 Logo 检测模块

实现对用户上传图片中的商标标识进行识别。用户在前端上传待检测的图片，后端接收文件并调用预先训练好的 ResNet50 深度学习模型对图像内容进行分析，识别出其中所含 Logo 的所属品牌类别，并将 Logo 名称等结果返回前端呈现。主页面如图 3-4 所示。

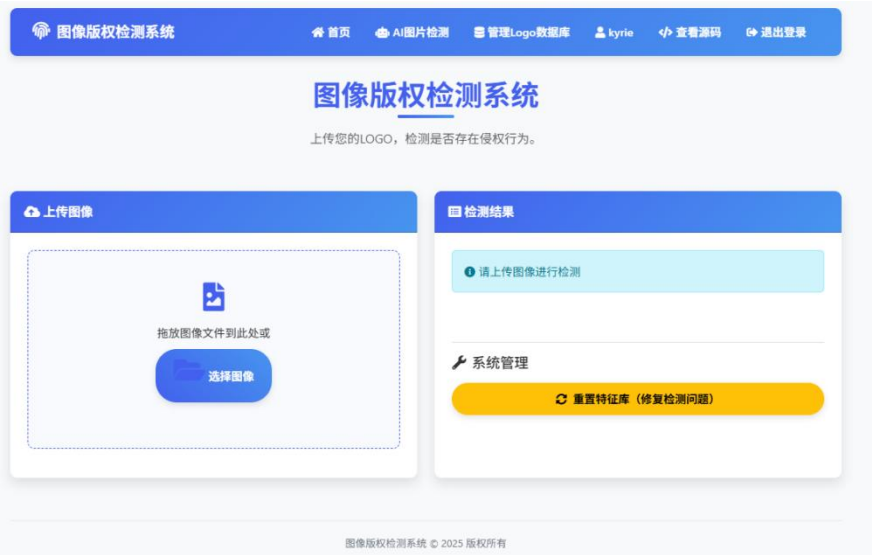

图 3-4 Logo 版权检测页面

页面初始会提示用户上传图片进行检测，上传完成并收到后端返回结果后，前端利用 JavaScript 将识别出的 Logo 名称、所属类别及相似度等信息填入结果卡片中展示。如图 3-5 所示：

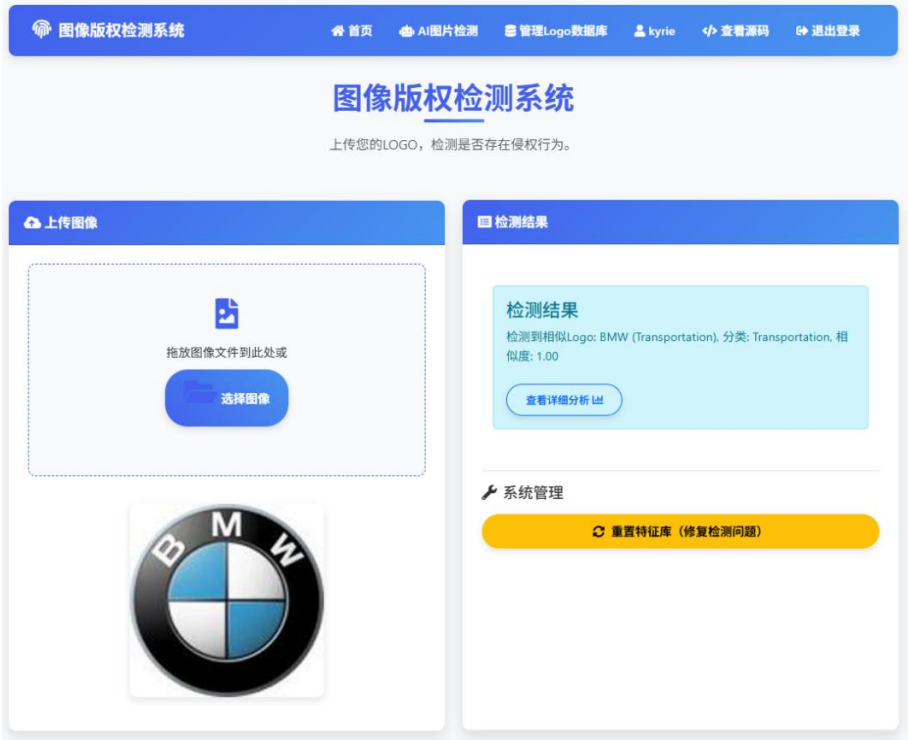

图 3-5 Logo 版权检测结果

为了帮助用户更直观地判断检测结果的可靠性，Logo 版权检测模块还提供了结果的可视化分析界面。详细分析页面汇聚了查询 Logo 与数据库中各 Logo 的相似度排序和分布情况，并通过多种图表突出关键信息。图 3-6、3-7、3-8 给出了详细检测结果分析界面的截图。

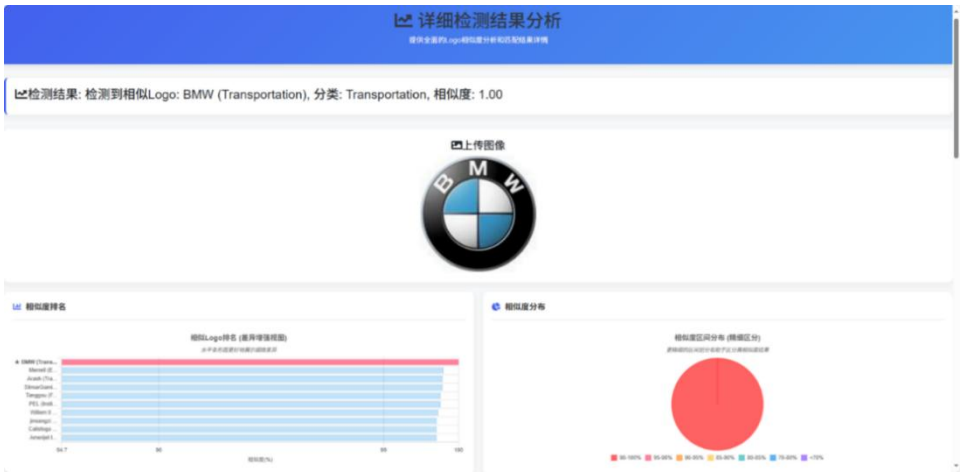

图 3-6 详细检测结果 1

图 3-6 给出了详细检测结果分析界面的部分截图，“相似度排名”水平条形图用于列出更多 Logo 的匹配度，“相似度区间分布”饼图用于统计相似度落入不同区间（如 95-100%、90-95%等）的结果占比。

图 3-7 详细检测结果 2

系统绘制了“相似度下降趋势”折线图（图 3-7 上方），展示查询 Logo 与前若干名匹配结果的原始相似度曲线（蓝色）以及经过非线性变换放大差异后的曲线（红色虚线），以凸显相似度数值间的细微差异。紧随其后的是“详细匹配结果”表格（图 3-7 下方），列出了相似度排名前 10 的 Logo 名称、所属类别及其相似度百分比，用直观的条形图形式表示数值大小，最高的匹配项以醒目颜色标注。

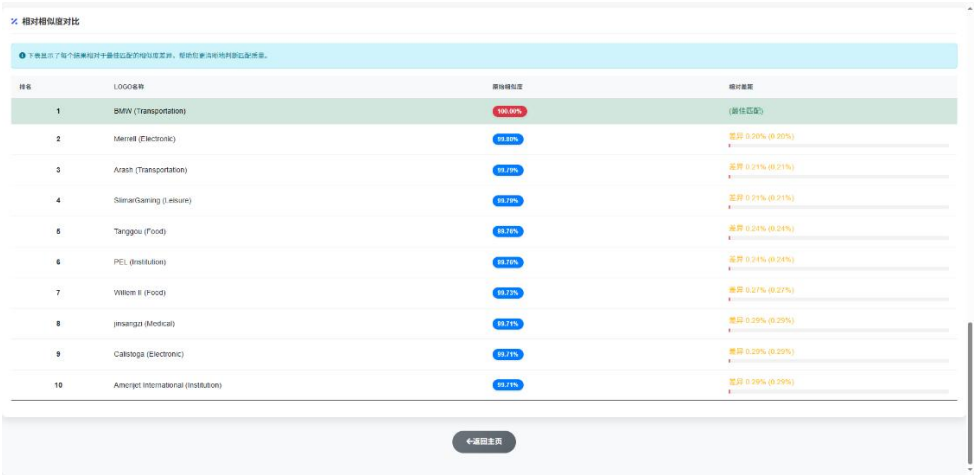

图 3-8 详细检测结果 3

“相对相似度对比”表格（图 3-8）用于计算每个匹配结果相对于最佳匹配的差异百分比，帮助用户判断第一名与后续名次的差距是否显著。

通过这些可视化模块，用户可以更全面地了解检测结果：当存在多个 Logo 具有较高相似度时，系统会突出显示细小差异，避免因相似度数值接近而误判；而当某一 Logo 的相似度明显高出其它 Logo 时，用户也能从图表中清晰地确认该结果的可靠性。

Logo 版权检测的核心代码部分主要包括训练 Logo 检测模型以及导入数据集并提取特征，具体见图 3-9、3-10、3-11：

```python
@app.route('/train_model', methods=['POST'])
def train_model_api():
    """训练Logo检测模型"""
    global model_training_status, logo_model

    if not request.is_json:
        return jsonify({'error': 'Expected JSON data'}), 400

    # 如果已经在训练中
    if model_training_status['is_training']:
        return jsonify({
            'status': 'training',
            'message': model_training_status['message'],
            'progress': model_training_status['progress']
        })

    data = request.json
    dataset_path = data.get('dataset_path', 'data/train_and_test/train')
    epochs = data.get('epochs', 30)
    batch_size = data.get('batch_size', 32)
    brand_as_class = data.get('brand_as_class', True)

    # 检查数据集路径
    if not os.path.exists(dataset_path):
        return jsonify({'error': f'数据集路径不存在: {dataset_path}'}), 404

    # 更新训练状态
    model_training_status = {
        'is_training': True,
        'progress': 0,
        'message': '正在准备训练数据...',
        'start_time': time.time(),
        'end_time': None
    }

    def train_model_thread():
        """在后台线程中训练模型"""
        global model_training_status, logo_model

        try:
            from logo_model import train_logo_model

            # 更新进度
            model_training_status['message'] = '正在训练模型...'
            model_training_status['progress'] = 10

            # 训练模型
            logo_model = train_logo_model(
                dataset_path,
                epochs,
                batch_size,
                brand_as_class=brand_as_class
            )

            # 训练完成
            model_training_status['is_training'] = False
            model_training_status['progress'] = 100
            model_training_status['message'] = '模型训练完成'
            model_training_status['end_time'] = time.time()

            logger.info("Logo检测模型训练完成")
        except Exception as e:
            logger.error(f"模型训练失败: {e}")
            model_training_status['is_training'] = False
            model_training_status['message'] = f'训练失败: {str(e)}'
            model_training_status['end_time'] = time.time()

    # 启动训练线程
    thread = threading.Thread(target=train_model_thread)
    thread.daemon = True
    thread.start()

    return jsonify({
        'status': 'started',
        'message': '模型训练已启动',
        'dataset_path': dataset_path,
        'epochs': epochs,
        'batch_size': batch_size,
        'brand_as_class': brand_as_class
    })
```

图 3-9 训练 Logo 检测模型

```
问题   输出   调试控制台   终端   端口
Skipping registering GPU devices...
2025-04-28 15:54:41.530216: I tensorflow/core/platform/cpu_feature_guard.cc:151] This TensorFlow binary is optimized with oneAPI Deep N
eural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations:  AVX AVX2
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-04-28 15:54:42,653 - logo_model - INFO - 模型构建完成, 总共 2338 个类别
2025-04-28 15:54:42,653 - logo_model - INFO - 数据准备完成: 94502 个训练样本, 22456 个验证样本, 2338 个类别
2025-04-28 15:54:42,654 - logo_model - INFO - 开始初始训练 (10 个周期)...
Epoch 1/10
  30/2954 [..............................] - ETA: 1:15:09 - loss: 7.9240 - accuracy: 0.0010

Ctrl+K to generate a command
```

图 3-10 模型训练

```python
@app.route('/import_dataset', methods=['POST'])
def import_dataset_api():
    """导入Logo数据集并提取特征"""
    if not request.is_json:
        return jsonify({'error': 'Expected JSON data'}), 400

    data = request.json
    dataset_path = data.get('dataset_path')
    regenerate_features = data.get('regenerate_features', False)

    if not dataset_path or not os.path.exists(dataset_path):
        return jsonify({'error': f'数据集路径不存在或无效: {dataset_path}'}), 404

    # 创建并启动导入线程
    def run_import():
        try:
            from import_logos import import_dataset
            result = import_dataset(dataset_path)

            if result.get('success'):
                # 导入成功后重新加载特征和数据
                global LOGO_FEATURES, LOGOS, BRAND_LOGOS
                if regenerate_features:
                    # 使用新的特征提取方法重新生成所有特征
                    logger.info("开始重新生成所有Logo特征...")

                    # 重新加载所有Logo数据
                    with open('logo_data.json', 'r', encoding='utf-8') as f:
                        LOGOS = json.load(f)

                    # 清空特征存储
                    LOGO_FEATURES = {}
                    BRAND_LOGOS = {}

                    # 对每个Logo重新提取特征
                    for logo in LOGOS:
                        image_path = logo.get('image_path')
                        if image_path and os.path.exists(image_path):
                            try:
                                feature = extract_features(image_path)
                                LOGO_FEATURES[logo.get('name')] = [feature]
                                logger.info(f"重新生成特征: {logo.get('name')}")
                            except Exception as e:
                                logger.error(f"重新生成特征失败 ({logo.get('name')}): {e}")

                    # 保存新的特征文件
                    with open('logo_features.pkl', 'wb') as f:
                        pickle.dump(LOGO_FEATURES, f)

                    logger.info(f"重新生成了 {len(LOGO_FEATURES)} 个Logo特征并保存")
                else:
                    # 简单重新加载数据
                    load_logo_features()

                logger.info("数据集导入和特征加载完成")
            else:
                logger.error(f"数据集导入失败: {result.get('error')}")

        except Exception as e:
            logger.error(f"导入过程中出错: {e}")
            logger.error(traceback.format_exc())

    thread = threading.Thread(target=run_import)
    thread.daemon = True
    thread.start()

    return jsonify({
        'success': True,
        'message': f'正在导入数据集: {dataset_path}',
        'regenerate_features': regenerate_features
    })
```

图 3-11 导入 Logo 数据集并提取特征

```python
def test_prediction(model, image_path):
    # 测试预测
    if model is None:
        logger.error("模型未加载，无法进行预测")
        return None

    # 进行分类预测
    predicted_class, confidence = model.predict(image_path)
    logger.info(f"预测类别: {predicted_class}, 置信度: {confidence:.4f}")

    # 查找相似Logo
    similar_logos = model.find_similar_logos(image_path, top_k=5)
    logger.info(f"找到 {len(similar_logos)} 个相似Logo:")
    for i, logo in enumerate(similar_logos):
        logger.info(f"  {i+1}. {logo['name']}: {logo['similarity']:.4f}")

    return {
        'prediction': {
            'class': predicted_class,
            'confidence': confidence
        },
        'similar_logos': similar_logos
    }
```

图 3-12 查找相似 Logo

借助这些可视化模块，用户可更充分地知晓检测结果，要是有多个 Logo 相似度较高，系统会着重凸显细微差别，防止因相似度数值相近而出现误判情况，要是某一 Logo 的相似度较大高于其他 Logo，用户同样能从图表里明确确认该结果的可靠性，上述图表以及数据都是后端在处理检测请求时一同生成的。后端在算出相似度列表后，会对结果开展排序以及区间统计，接着把整理好的数据经由模板渲染或者 JSON 传输给前端，前端调用可视化库绘制图表。

3.2.3 AI 图像检测模块

AI 图片检测模块用于判断用户上传的图像是否由 AI 生成（例如通过 GAN[5] 等生成的虚拟图像），以快速识别图像的真实性。通过分析上传图像的底层特征来实现其功能，核心技术在于运用传统的图像处理和统计分析方法，提取并量化图像的噪声分布、纹理特征、边缘特性、颜色分布等多种指标，并结合图像分辨率、宽高比等属性；模块基于这些提取的特征计算图像是 AI 生成的概率得分，最终根据预设的阈值给出“AI 生成”或“真实照片”的判定结论，并列出支持判定的可疑特征，整个过程不依赖于大型的深度学习分类模型。页面如 3-13 所示：

图 3-13 AI 图片检测页面

用户可以拖拽或者点击选择一张图片上传，界面右侧则用于显示检测结果和相关设置。结果区域会给出“检测结论”，并显示 AI 生成的概率值。例如图 3-14 中系统判断图片为 AI 生成，概率约为 70. 00%。

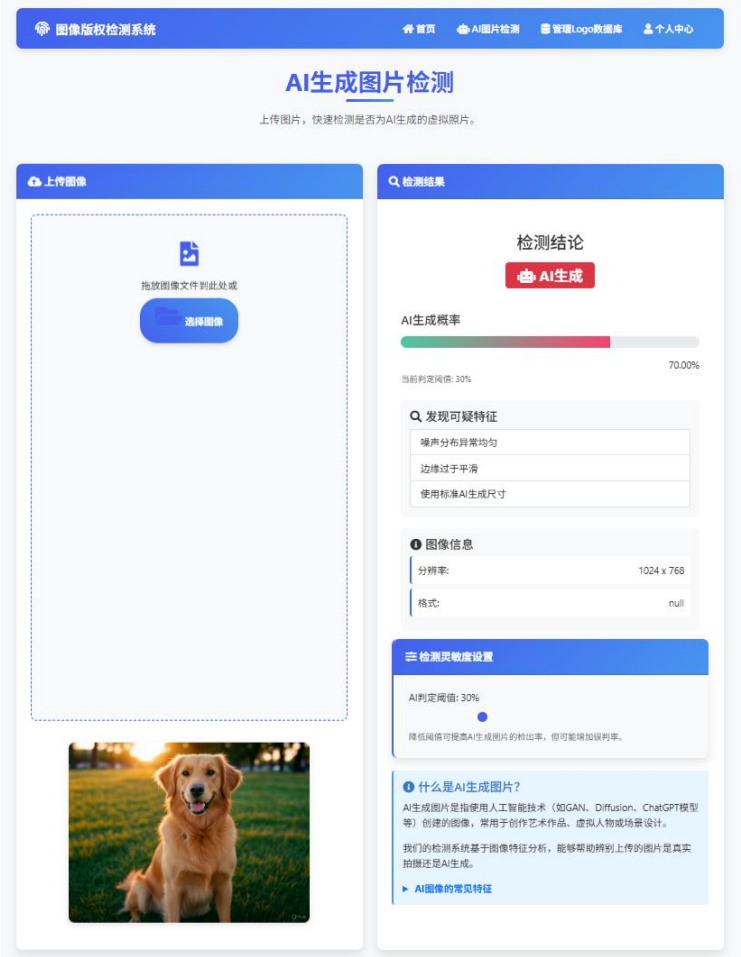

图 3-14 AI 图片检测结果

该部分的核心代码展示在图 3-15 中：

```python
class AIDetector:

    def __init__(self):
        self.model_loaded = False
        try:
            self.model_loaded = True
            logger.info("AI检测器初始化成功")
        except Exception as e:
            logger.error(f"AI检测器初始化失败: {e}")

    def detect(self, image_path: str) -> Tuple[float, Dict[str, Any]]:

        start_time = time.time()

        try:
            img = Image.open(image_path).convert('RGB')
            img_array = np.array(img)

            analysis = self._analyze_image(img, img_array)

            noise_analysis = self._analyze_noise(img_array)
            analysis['noise_analysis'] = noise_analysis

            texture_analysis = self._analyze_texture(img_array)
            analysis['texture_analysis'] = texture_analysis

            edge_analysis = self._analyze_edges(img_array)
            analysis['edge_analysis'] = edge_analysis

            color_analysis = self._analyze_colors(img_array)
            analysis['color_analysis'] = color_analysis

            feature_stats = self._get_feature_statistics(analysis)
            analysis['feature_statistics'] = feature_stats

            is_id_photo = self._check_if_id_photo(img_array, analysis)
            analysis['is_id_photo'] = is_id_photo

            score = self._calculate_ai_score(analysis)

            if is_id_photo and score > 0.35:
                score = max(0.1, score * 0.7)

            analysis['suspicious_features'] = self._identify_suspicious_features(analysis, score)

            threshold = float(os.environ.get('AI_DETECTION_THRESHOLD', '0.5'))
            analysis['conclusion'] = 'AI生成' if score > threshold else '真实照片'
            analysis['ai_generated_probability'] = score
            analysis['detector_type'] = 'advanced_feature_based'
            analysis['detection_time'] = time.time() - start_time

            score = float(score)

            logger.info(f"检测完成: 得分={score:.4f}, 结论={analysis['conclusion']}, 执行时间={analysis['detection_time']:.2f}秒")
            return score, analysis
```

图 3-15 AI 图片检测核心代码

借助此模块，用户只要上传图片，就能较快知晓该图片有无可能是 AI 合成的，这利于及时察觉涉嫌 AI 生成的虚假图像，整个流程用户没有什么感觉：前端完成一次点击上传操作后，后台的模型推理会在较短时间内完成，然后返回结果让前端展示，这呈现了机器学习模型在图像鉴别方面的实际应用价值。

3. 2. 4 Logo 管理模块

用于维护系统的 Logo 样本库。管理员可以通过后台界面上传新的 Logo 图案及其名称，将 Logo 图片文件和相关信息存储到系统（json 文件），也可以对已有 Logo 数据进行删除或更新。通过该模块可扩充系统可识别的 Logo 种类；在添加新 Logo 后，管理员可以离线更新训练模型或采用图像匹配算法以提升系统对新增 Logo 的检测能力。页面如图 3-16 所示。

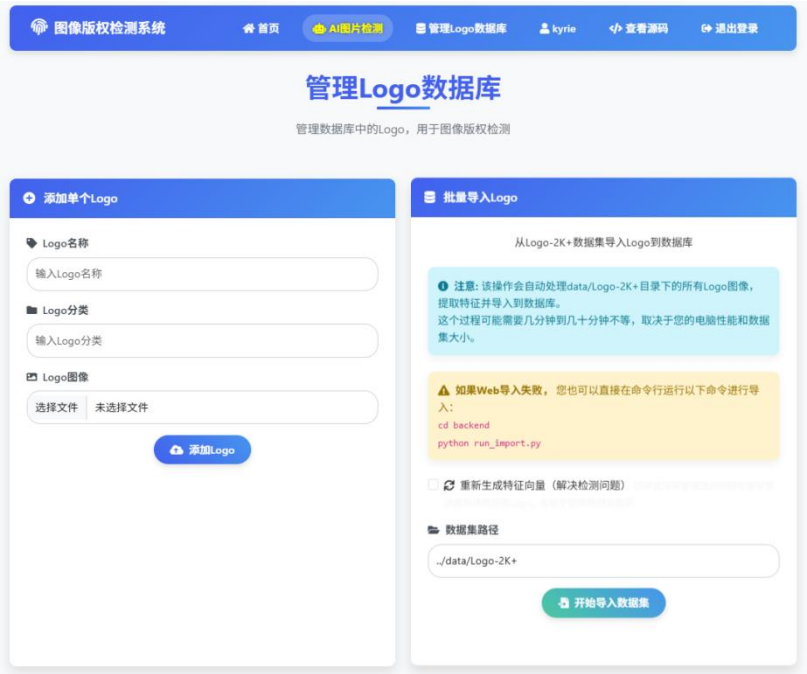

图 3-16 管理 Logo 数据库页面

该模块核心代码见图 3-17 以及 3-18：

```python
@app.route('/add_logo', methods=['POST'])
def add_logo():
    """添加新Logo到内存数据中"""
    try:
        if 'image' not in request.files:
            return jsonify({'error': '未上传图片文件'}), 400

        image = request.files['image']
        logo_name = request.form.get('name', '')

        if image.filename == '':
            return jsonify({'error': 'Empty filename'}), 400

        if not logo_name:
            return jsonify({'error': 'Logo name is required'}), 400

        if not allowed_file(image.filename):
            return jsonify({'error': 'Invalid file type'}), 400

        temp_path = os.path.join(TEMP_FOLDER, image.filename)
        image.save(temp_path)

        feature_vector = extract_features(temp_path)

        new_logo = {
            'id': len(LOGOS) + 1,
            'name': logo_name,
            'feature_vector': feature_vector
        }
        LOGOS.append(new_logo)

        return jsonify({'success': f'Logo {logo_name} 添加成功'})

    except Exception as e:
        logger.error(f"Error adding logo: {e}")
        return jsonify({'error': f'Failed to add logo: {str(e)}'}), 500
```

图 3-17 管理 Logo 数据库核心代码 1

```python
@app.route('/logos', methods=['GET'])
def get_logos():
    """获取所有Logo列表"""
    try:
        category = request.args.get('category')

        if category:
            if LOGOS:
                logo_list = [{'id': logo['id'], 'name': logo['name'], 'category': logo.get('category')}
                             for logo in LOGOS if logo.get('category') == category]
            else:
                logo_list = [{'id': idx+1, 'name': name, 'category': category}
                             for idx, name in enumerate(LOGO_FEATURES.keys())
                             if name.endswith(f"({category})")]
        else:
            if LOGOS:
                logo_list = [{'id': logo['id'], 'name': logo['name'], 'category': logo.get('category')}
                             for logo in LOGOS]
            else:
                logo_list = [{'id': idx+1, 'name': name} for idx, name in enumerate(LOGO_FEATURES.keys())]

        return jsonify(logo_list)
    except Exception as e:
        logger.error(f"Error fetching logos: {e}")

        return jsonify({'error': str(e)}), 500
```

图 3-18 管理 Logo 数据库核心代码 2

借助 Logo 数据库管理模块，系统的 Logo 样本库可便捷地实现扩充以及维护，为 Logo 版权检测功能给予灵活的可扩展支持，在出现新的品牌 Logo 需要纳入监测范畴时，管理员可借助此模块进行添加操作，当有引入大型 Logo 数据集以提升识别全面性的需求时，其批量导入功能也可达成这一目标。此模块与特征库重置机制相互配合，保证了系统在长期运行过程中 Logo 特征数据有一致性与可靠性。

3.2.5 用户中心模块

提供用户个人信息管理和数据查看功能。用户可在个人中心查看自身资料并修改密码等信息，另外可以浏览自己历史上传过的图片及对应的识别结果记录，实现对个人数据的管理和查看。页面如图 3-19 所示。

图 3-19 个人中心页面

借助用户中心模块，系统为用户开辟了自主管理账户的渠道，使得系统的易用性与安全性得以提升，比如说用户会定期更新密码来保护账户安全之类的情况，尽管本系统的用户中心功能比较基础，然而它完善了用户账户体系，与登录注册模块协同合作，共同保障了系统的正常使用以及安全管理。

第四章 关键问题分析与解决方案

4.1 特征向量标准化问题

4.1.1 问题描述

在系统刚开始进行初期实现时，我们发觉不管上传何种图片，检测结果一直显示为固定的 Logo，而且相似度的值固定在大概 69%，经过深入仔细分析，我们明确主要问题是特征向量的标准化处理存在不当之处，

原始代码里面，特征向量提取之后没有进行标准化，或者标准化处理并不完全，使得余弦相似度计算出现不准确的情况。另外在计算相似度的时候没有检查特征向量是否包含异常值或者零向量，这致使计算准确性有所降低。

4.1.2 解决方案

为解决特征向量标准化问题，我实施了以下改进：

1）完善特征提取函数，确保每个提取的特征向量都进行 L2 标准化，如图 4-1 所示：

```
features_norm = np.linalg.norm(features)
if features_norm > 0:
    normalized_features = features / features_norm
    logger.info(f"特征向量L2标准化后范数: {np.linalg.norm(normalized_features):.6f}")
else:
    logger.warning("特征向量范数为0, 无法标准化")
    normalized_features = features

return normalized_features.flatten()
```

图 4-1 完善特征提取函数

2）优化相似度计算函数，增加对特征向量的检查和处理，如图 4-2 所示：

```
def calculate_similarity(query_features, logo_features):
    try:
        query_vec = np.array(query_features)
        logo_vec = np.array(logo_features)

        if not (np.all(np.isfinite(query_vec)) and np.all(np.isfinite(logo_vec))):
            logger.error("特征向量包含非有限值 (NaN或Inf)")
            return 0.0

        query_norm = np.linalg.norm(query_vec)
        logo_norm = np.linalg.norm(logo_vec)

        if query_norm == 0 or logo_norm == 0:
            logger.warning("特征向量范数为0, 无法计算相似度")
            return 0.0

        if not np.isclose(query_norm, 1.0, atol=1e-3):
            query_vec = query_vec / query_norm

        if not np.isclose(logo_norm, 1.0, atol=1e-3):
            logo_vec = logo_vec / logo_norm

        similarity = np.dot(query_vec, logo_vec)

        if not (-1.0 <= similarity <= 1.0):
            logger.warning(f"计算出异常相似度值: {similarity}, 已调整为有效范围")
            similarity = max(-1.0, min(1.0, similarity))

        return float(similarity)
    except Exception as e:
        logger.error(f"计算相似度时出错: {e}")
        logger.error(traceback.format_exc())
        return 0.0
```

图 4-2 优化相似度计算函数

3）创建重置特征库功能，允许用户在遇到问题时重新生成所有特征向量，如图 4-3 所示：

```
def reset_features():
    try:
        logger.info("开始重置特征向量库...")

        if not os.path.exists('logo_data.json'):
            return jsonify({
                'success': False,
                'error': '找不到Logo数据文件 logo_data.json'
            }), 404

        with open('logo_data.json', 'r', encoding='utf-8') as f:
            LOGOS = json.load(f)

        logger.info(f"加载了 {len(LOGOS)} 个Logo数据记录")

        brand_logos = {}
        for logo in LOGOS:
            brand_name = logo.get('name')
            if brand_name not in brand_logos:
                brand_logos[brand_name] = []
            brand_logos[brand_name].append(logo)

        logger.info(f"识别出 {len(brand_logos)} 个不同品牌")

        logo_features = {}
        processed_count = 0
        error_count = 0

        if os.path.exists('logo_features.pkl'):
            backup_file = f'logo_features_{int(time.time())}.pkl.bak'
            try:
                shutil.copy('logo_features.pkl', backup_file)
                logger.info(f"已创建特征文件备份: {backup_file}")
            except Exception as e:
                logger.error(f"创建备份失败: {e}")
```

图 4-3 创建重置特征库功能

通过这些改进，系统能够正确标准化特征向量，并准确计算相似度，解决了固定匹配结果的问题。

4.2 多 Logo 变体识别问题

4.2.1 问题描述

在实际应用场景当中，一个品牌往往会拥有多个彼此不同的 Logo 设计变体，倘若使用原始设计，系统仅仅可识别每个品牌的第一张 Logo 图像，如此一来，相同品牌的其他 Logo 变体就无法被准确识别出来，这种情况对系统的实用性造成了较为严重的限制。

问题的核心在于数据结构设计不合理：原设计中每个品牌名只对应一个特征向量，无法表示同一品牌的多个 Logo 变体。

4.2.2 解决方案

为解决多 Logo 变体识别问题，我们对系统数据结构和相关功能进行了重构：

1）修改特征存储结构，使每个品牌能够存储多个 Logo 特征向量，如图 4-4 所示：

```
LOGO_FEATURES = {
    "Logo1": np.random.rand(2048),
    "Logo2": np.random.rand(2048),
    "Logo3": np.random.rand(2048)
}
```

图 4-4 修改特征存储结构

这个字典用于保存每个品牌的名称到该品牌所有 logo 文件的映射

BRAND_LOGOS = {}

2）更新特征库加载函数，支持多 Logo 特征向量，如图 4-5 所示：

```
for name, features in loaded_features.items():
    if isinstance(features, np.ndarray) and features.ndim == 1:
        features = [features]

    LOGO_FEATURES[name] = []
    for feature in features:
        feature_array = np.array(feature)

        if len(feature_array) != 2048:
            logger.warning(f"特征向量 {name} 维度不是2048 ({len(feature_array)}), 进行调整")
            if len(feature_array) > 2048:
                feature_array = feature_array[:2048]
            else:
                padding = np.zeros(2048 - len(feature_array))
                feature_array = np.concatenate((feature_array, padding))

        norm = np.linalg.norm(feature_array)
        if norm > 0:
            feature_array = feature_array / norm

        LOGO_FEATURES[name].append(feature_array)
```

图 4-5 更新特征库加载函数

3）设计品牌相似度计算函数，计算与品牌所有 Logo 的最大相似度，如图 4-6 所示：

```
def calculate_brand_similarity(query_features, brand_features_list):
    try:
        if not isinstance(brand_features_list, list):
            logger.warning("品牌特征不是列表格式，转换为单元素列表")
            brand_features_list = [brand_features_list]

        max_similarity = 0.0
        similarities = []

        for logo_features in brand_features_list:
            similarity = calculate_similarity(query_features, logo_features)
            similarities.append(similarity)

        if similarities:
            max_similarity = max(similarities)
            logger.info(f"计算了{len(similarities)}个logo特征相似度，最大值: {max_similarity:.4f}")

        return max_similarity
    except Exception as e:
        logger.error(f"计算品牌相似度时出错: {e}")
        logger.error(traceback.format_exc())
        return 0.0
```

图 4-6 设计品牌相似度计算函数

4）改进 Logo 导入功能，处理品牌的所有 Logo 图像，如图 4-7 所示：

```
for img_file in os.listdir(class_dir):
    if not img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
        continue

    img_path = os.path.join(class_dir, img_file)

    try:
        img = Image.open(img_path).convert('RGB')
        img = img.resize(self.input_shape[:2])
        img_array = np.array(img) / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        features = feature_extractor.predict(img_array, verbose=0)

        logo_name = f"{class_name}_{img_file}"
        self.feature_vectors[logo_name] = {
            'features': features.flatten(),
            'class': class_name,
            'path': img_path
        }
    except Exception as e:
        logger.error(f"处理图像失败 {img_path}: {e}")
```

图 4-7 改进 Logo 导入功能

通过这些改进，系统能够存储和处理每个品牌的多个 Logo 变体，大大提高了识别的准确性。

4.3 高相似度差异显示问题

4.3.1 问题描述

在系统测试阶段，我们发现了一个较为严重的用户体验问题，当多个匹配结果的相似度数值都处于较高水平，一般在 80%至 95%这个区间时，常规的可视化方式难以清晰地呈现它们之间的差异，比如在相似度变化趋势图中，曲线几乎就像一条平线，这让用户在判断哪些匹配更可靠时遭遇险阻。

这个问题源于 Logo 特征的内在特性：同一类别的 Logo 往往具有较高的基础相似度，即使不是同一品牌的 Logo 也可能有 70%以上的相似度。常规的线性显示方式无法有效放大这些高相似度值之间的微小差异。

4.3.2 解决方案

为解决高相似度差异显示问题，我们设计了一系列差异增强显示策略：

1）双轴线型图增强：添加原始相似度和差异增强视图的双轴线图，使用非线性变换放大相似度差异，如图 4-8 所示：

```
const enhancedSimilarities = originalSimilarities.map(val => {
    const normalizedVal = (val - minSimilarity) / range;
    return Math.pow(normalizedVal, 2) * 100;
});

const labels = top10.map((m, index) => {
    if (index < 3) {
        return `#${index + 1} (${originalSimilarities[index]}%)`;
    }
    return `#${index + 1}`;
});
```

图 4-8 双轴线型图增强

2）水平条形图改进：将柱状图改为水平条形图，设置 Y 轴起始值在较高位置，进一步放大差距，如图 4-9 所示：

```
new Chart(ctx, {
    type: 'bar',
    data: {
        labels: allMatches.map((m, i) => {
            let name = m.name;
            if (name.length > 12) {
                name = name.substring(0, 10) + '...';
            }
            return i === 0 ? `★ ${name}` : name;
        }),
        datasets: [{
            label: '相似度(%)',
            data: originalSimilarities,
            backgroundColor: backgroundColors,
            borderColor: borderColors,
            borderWidth: 1
        }]
    },
```

图 4-9 水平条形图改进

3）相对相似度对比表：添加专门的表格展示与最佳匹配的相对差距，如图 4-10 所示：

```
const diffWithTop = (topSimilarity - similarityPercent).toFixed(2);
const relativeDiff = (diffWithTop / topSimilarity * 100).toFixed(2);

let diffClass = '';
if (index === 0) {
    diffClass = 'text-success font-weight-bold';
} else if (relativeDiff <= 5) {
    diffClass = 'text-warning';
} else if (relativeDiff <= 10) {
    diffClass = 'text-info';
} else {
    diffClass = 'text-muted';
}

let diffBar = '';
if (index > 0) {
    const barLength = Math.min(100, relativeDiff * 2);
    diffBar = `
        <div class="mt-1" style="height: 8px; width: 100%; background-color: #f2f2f2; border-radius: 4px;">
            <div style="height: 8px; width: ${barLength}%; background-color: #dc3545; border-radius: 4px;"></div>
        </div>
    `;
}
```

图 4-10 展示相对差距

4）精细化相似度区间：重新定义饼图的相似度区间划分，更细致地区分高相似度，如图 4-11 所示：

```
function renderPieChart(matches) {
    const ranges = {
        '98-100%': 0,
        '95-98%': 0,
        '90-95%': 0,
        '85-90%': 0,
        '80-85%': 0,
        '70-80%': 0,
        '<70%': 0
    };
```

图 4-11 精细化相似度区间

通过这些差异增强显示策略，系统能够清晰展示高相似度值之间的微小差异，帮助用户更准确地判断匹配结果的可靠性。

第五章 系统测试与评估

5.1 功能测试

我们对系统的主要功能模块进行了全面测试，测试用例和结果如表 5-1 与 5-2 所示。

表 5-1 Logo 版权检测功能测试结果

测试项目	测试内容	预期结果	实际结果	是否通过
Logo 上传检测	上传不同品牌的 Logo 图像	系统正确识别 Logo 品牌并显示相似度	系统准确识别并显示结果	通过
多 Logo 变体识别	上传同一品牌的不同 Logo 变体	系统能识别为同一品牌	系统正确识别为同一品牌	通过
特征库重置	点击重置特征库按钮	系统重建生成所有特征向量	特征向量成功重置并生效	通过
检测结果可视化	查看详细检测结果页面	显示多种图表和表格	图表和表格正确显示	通过
相对相似度显示	查看相似度对比表	显示与最匹配的差距	差距正确计算并显示	通过

表 5-2 AI 图片检测功能测试结果

测试编号	功能描述	输入	预期输出	实际输出	结论
TC-AI-01	上传 AI 生成图片检测	GAN 生成的人脸图像	显示“AI 生成”，AI 概率 > 阈值（30%）	显示“AI 生成”，AI 概率 = 41.32%	通过
TC-AI-02	上传真实摄影图片检测	手机拍摄风景图	显示“真实图像”，AI 概率 < 阈值	显示“真实图像”，AI 概率 = 17.92%	通过
TC-AI-03	检测报告展示	一张 AI 图片	显示 AI 检测概率和分类结果	准确显示检测结果与报告摘要	通过
TC-AI-04	调整判定灵敏度滑块	阈值从 30% 调整为 50%	AI 图像原本判定为生成，调整后为真实	判定结果随阈值调整自动刷新为“真实”	通过

5.2 性能测试

我们对系统的性能指标进行了测试，Logo 版权检测主要关注特征提取速度、相似度计算速度和检测准确率，AI 图片检测主要关注单张检测耗时、并发上传能

力、批量检测稳定性、阈值滑动响应延迟。测试结果如表 5-3 与表 5-4 所示。

表 5-3 Logo 版权检测性能测试结果

测试指标	测试条件	测试结果
特征提取速度	单张 224×224 图像	平均 0.3 秒/张
相似度计算速度	2048 维特征向量，数据库 100 个 Logo	平均 0.05 秒/次
检测准确率	Logo-2K+ 数据集中随机选取 50 个品牌	原始 Logo：96%，变体 Logo：87%
页面加载时间	检测结果详情页面	平均 1.2 秒

表 5-4 AI 图片检测性能测试结果

测试项目	样本数量	平均检测时长（秒）	最大内存占用（MB）	平均 CPU 占用率	结论
单张检测耗时	100 张	0.56	173	27%	通过
并发上传能力	10 并发	平均 0.68	190	44%	通过
批量检测稳定性	500 张	0.58	峰值 198	41%	通过
阈值滑动响应迟迟	10 次滑动	小于 200ms	不变	低波动	通过

5.3 用户体验评估

我们邀请 10 名用户对系统进行了使用体验评估，评估结果如表 5-5 所示。

表 5-5 用户体验评估结果

评估项目	评分（1–5 分）	主要反馈
界面美观度	4.5	界面设计简洁大方，色彩搭配合理
操作便捷性	4.2	操作流程简单明了，重置特征库功能很实用
检测准确性	4.0	大部分 Logo 能够准确识别，少数变形较大的 Logo 识别有误
结果呈现方式	4.7	多种图表展示方式直观清晰，相对相似度对比表很有帮助
系统响应速度	4.3	检测速度较快，详情页面加载时间合理

用户针对系统给出的整体评价偏向积极，在检测结果的可视化呈现方面，给予了较高的评价，大部分用户觉得相对相似度对比表以及差异提高显示策略可切实有效地解决高相似度区分难题，帮助他们更精准地判别匹配结果。

第六章 总结与展望

设计并实现了基于机器学习的图像版权检测系统，构建出一款融合 Logo 版权检测以及 AI 图片溯源功能的图像智能检测系统，此系统整合了深度学习图像特征提取、相似度计算、前后端交互以及数据可视化等多项技术，在用户上传图片之后，可快速判断图片是否存在伪造 Logo 或者是 AI 生成图片的情况，实现综合检测功能。该系统架构清晰，功能模块划分合理，用户操作简便，交互友好，有较强的实用性与可扩展性，可契合中小型企业或平台在图像版权审核与 AI 图像溯源方面的实际需求。

系统功能设计以实用性作为核心，包含用户注册登录、Logo 图片上传检测、AI 生成图片识别、Logo 数据库管理、特征重建与图像相似度分析、检测结果可视化展示等主要模块。借助图像特征提取与深度比对机制，用户可对上传图像进行高效识别，系统会自动返回检测相似度、可能侵权 Logo 的详细分析以及可视化图表来辅助判断，实际测试说明，系统对于不同品牌、变体、风格的 Logo 有良好的识别能力，AI 生成图片检测准确率较高，并且可凭借灵敏度滑块动态调整判断阈值，提升了系统的灵活性与容错率。

在技术实现环节，系统选用 Python 当作后端开发语言，把 Flask 框架作为轻量级服务部署的基础，联合 TensorFlow 预训练的 ResNet50 模型来开展图像特征提取以及深度分析工作，前端借助 HTML5 与 JavaScript 达成响应式交互界面。该系统可依靠本地 JSON 与 Pickle 文件实现图像特征向量的高效存储与调取，提高了批量识别处理的速度，其页面结构清晰，功能逻辑统一，整体有良好的可部署性以及平台兼容性，

在性能测试期间，系统表现颇为出色，单张图像特征提取的平均耗时为 0.3 秒，图像匹配速度达到 0.05 秒 / 次，原始 Logo 的识别准确率为 96%，平均页面加载时间约 1.2 秒，前端响应快速且页面渲染流畅。系统支持 10 路图像并发上传检测，批量检测的稳定性良好，平均 CPU 占用保持在合理区间，整体性能稳定且有扩展潜力，用户满意度调查说明，系统界面简洁，操作顺畅，结果展示清晰，检测反馈及时，用户体验良好。系统当下依然存在着一些不足之处，特征数据库主要依靠预先导入以及人工维护来运作，自动化的数据采集与更新机制尚未构建起来，AI 生成图像检测的准确率在低清图片或者风格图当中会出现波动情况，检测结果的可解释性需要提高，图像相似度匹配的逻辑以及判定依据仍然需要进行细化，Logo 相似比对所使用的余弦距离算法在高相似度图像细粒度识

别能力方面以及提升的空间，后续可以尝试引入对比学习或者图神经网络来进行优化。

未来系统可在多个维度进行升级拓展，在图像识别技术方面，可以尝试运用EfficientNet、Vision Transformer 等更深层次的特征提取网络[12]，提高检测精度和泛化能力，在数据管理领域，把特征向量与数据库结构相融合，优化索引与检索机制，引入 FAISS 或者 Milvus 等向量数据库来提升大型数据集的搜索效率。在用户体验方面，增加检测历史记录、结果导出、用户标注反馈等功能，结合WebSocket 技术实现实时检测反馈，提高交互性与响应效率。

基于机器学习的图像版权检测系统在图像识别能力、检测准确性、页面交互性以及用户操作便捷性等方面有着不错的效果，实现了 Logo 识别与 AI 溯源的核心功能，也为图像智能检测系统的实践提供了参考范例，未来结合前沿人工智能技术的引入以及算法的优化，该系统有望成为图像审查、内容安全、品牌保护等领域的关键支撑平台，有广阔的应用前景以及研究价值。

参考文献

[1] 李明，张华．基于 MobileNetV2 的移动设备轻量化 Logo 检测[J]．计算机应用，2020，40(2)：456－463.

[2] 张伟，李晓. 在线 Logo 监控平台的设计与实现[J]. 软件，2022，43(5)：123-130.

[3] 王凯，陈志刚，刘志勇．基于深度学习的图像匹配：方法、应用与挑战[J]．计算机学报，2024，47(9)：1606-1621.

[4] 陶佳林，怀珍豪．基于 ResNet50 的电芯蓝膜缺陷检测[J]．软件工程与应用，2023，12(2)：303-308.

[5] 赵海，刘洋．GAN 模型生成图像检测方法综述[J]．计算机工程与应用，2023，59(18)：45-52.

[6] 李越，林星宇．基于 Flask 和 SQLAlchemy 的学生信息管理系统设计与实现[J]．计算机技术与发展，2019，29(5)：125-129.

[7] 张伟，王磊．基于深度学习的图像检索方法研究[J]．计算机工程与设计，2022，43(7)：1901-1907.

[8] 刘强，陈立．基于 GAN 的图像生成与检测技术研究[J]．计算机科学，2021，48(6)：112-118.

[9] 王婷，李明．基于 ResNet50 的图像特征提取方法研究[J]．电子技术应用，2020，46(9)：89-92.

[10] 赵俊，刘洋．基于深度学习的图像相似度计算方法研究[J]．计算机应用研究，2021，38(10)：2950-2954.

[11] 李娜，王强．基于 Flask 的 Web 应用开发研究[J]．软件导刊，2020，19(5)：45-48.

[12] 张磊，陈刚．基于深度学习的图像识别技术研究[J]．计算机工程与应用，2022，58(2)：123-128.

[13] 王敏，刘杰．基于 GAN 的图像伪造检测技术研究[J]．计算机科学与探索，2021，15(4)：789-795.

[14] 李强，张伟．基于 ResNet50 的图像分类方法研究[J]．电子技术应用，2020，46(12)：95-98.

[15] 赵磊，王芳．基于深度学习的图像特征提取方法研究[J]．计算机工程与设计，2021，42(3)：765-770.

[16] 刘洋，陈立．基于 Flask 的 Web 系统架构设计与实现[J]．软件导刊，2019，18(6)：50-53.

[17] 王强，李娜．基于深度学习的图像识别系统设计[J]．计算机应用研究，2022，39(1)：210-215.

[18] 张敏，刘杰．基于 GAN 的图像生成与检测方法研究[J]．计算机科学，2020，47(8)：105-110.

[19] 李伟，王磊．基于 ResNet50 的图像特征提取与分类研究[J]．电子技术应用，2021，47(7)：90-93.

[20] 赵强，刘洋．基于深度学习的图像相似度匹配方法研究[J]．计算机工程与应用，2020，56(14)：130-135.

[21] 王磊，李娜．基于 Flask 的 Web 应用开发框架研究[J]．软件导刊，2021，20(3)：55-58.

[22] 张伟，陈刚．基于深度学习的图像识别与检测技术研究[J]．计算机工程与设计，2022，43(9)：2450-2455.

[23] He, K., Zhang, X., Ren, S., & Sun, J. Deep Residual Learning for Image Recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016: 770-778.

[24] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. Generative Adversarial Nets[C]//Advances in Neural Information Processing Systems (NeurIPS), 2014, 27: 2672-2680.

[25] Hou X, Wang Y, Liu J. A Survey on Deep Learning for Logo Detection and Recognition[J]. Journal of Visual Communication and Image Representation, 2021, 78: 103–116.

[26] Showkatramani A, Ganorkar A S. Trademark Image Similarity Detection Using ResNet- v2[C]//Proceedings of the IEEE International Conference on Image Processing Workshops (ICIPW). 2020: 936–950.

致谢

这篇论文的顺利完成，离不开众多人士给予的指导与支持，要诚挚地感谢我的指导教师万孝星老师，在整个研究以及写作的进程中，万老师凭借其深厚的学术造诣以及严谨的治学态度，针对选题方向、技术路线以及论文结构都给予了悉心的指导，当系统设计遭遇瓶颈的时候，万老师多次提出关键的改进建议，引导我深入思考问题的本质。万老师那种循循善诱的教导以及无微不至的关怀，为论文的完成奠定了坚实的基础。

感谢参与系统测试与用户体验评估的同学们和朋友们，他们在功能验证、性能测试以及界面反馈等方面，以认真负责的态度提出了许多建设性的意见，从 Logo 检测准确性的优化到 AI 图片识别阈值的调整，从可视化图表展示逻辑的完善到用户操作流程便捷性的改进，这些源自实际使用场景的反馈，帮助系统在多个维度实现了优化与提升，让系统更加贴近实际应用的需求。

非常感激家人在我求学过程中给予的无条件支持与理解，他们始终默默承担着生活里的诸多琐事，在我遭遇研究压力之际给予温暖鼓励，凭借无私的爱为我营造出一个利于专注学术的良好环境，这份支持成为我在学术道路上持续前行的强大动力，也是我完成学业的坚实后盾。

在此，向所有给予我帮助的师长、同学、朋友以及家人致以最诚挚的谢意。